ホモトピー型理論

上村 太一

2023 年 3 月 14 日

「ホモトピー型理論」

© 2023 上村 太一

HTML 版: https://uemurax.github.io/hott-ja/index.html
PDF 版: https://uemurax.github.io/hott-ja/index.pdf
リポジトリ: https://github.com/uemurax/hott-ja

この作品はクリエイティブ・コモンズ表示 4.0 国際ライセンス[*1]の下に提供されています。

[*1] http://creativecommons.org/licenses/by/4.0/

目次

第 1 章　はじめに [0000]　1
1.1　識別子 [000R]　1

第 I 部　基礎 [0006]　3

第 2 章　型理論 [0001]　5
2.1　宇宙 [0009]　5
2.2　関数型 [000A]　6
2.3　レコード型 [000B]　7
2.4　同一視型 [000C]　10
2.4.1　高次グルーポイド構造 [002N]　12
2.5　自然数 [002T]　13
2.6　有限余積 [002U]　14

第 3 章　一価性公理 [0002]　17
3.1　可縮性 [000S]　17
3.2　同一視型の基本定理 [001R]　20
3.3　一価性 [000X]　23
3.4　関数外延性 [001Y]　24
3.4.1　一価性から関数外延性を導く [0021]　25
3.5　構造同一原理 [0022]　25
3.6　同値 [000U]　27
3.6.1　他の同値の概念 [004I]　30

第 4 章　高次帰納的型 [0003]　33
4.1　始代数 [0031]　33
4.2　円周 [0032]　36
4.3　押し出し [0033]　36

4.4 降下性 [0034] 37

第 5 章 n 型 [003W] 41
5.1 命題 [003Z] 43
5.1.1 同値の概念 [004N] 44
5.2 集合 [004B] 46
5.3 切り詰め [004Z] 47
5.4 述語論理 [0057] 48
5.5 構造同一原理 [004V] 49

第 II 部 数学 [0007] 53

第 6 章 ホモトピー論 [0005] 55

第 7 章 圏論 [0004] 57

参考文献 59

索引 61

識別子一覧 63

図目次

第 1 章

はじめに [0000]

1.1 識別子 [000R]

本書ではすべての章や定理などに一意で恒久的な**識別子**が割り当てられている。識別子は 4 桁の英数字で、題名の横に例えばの [05AF] ように表示される。章番号や定理番号は本書の更新に伴って変わることがあるのに対し、識別子は変わることはない。そのため、本書の特定の箇所を参照する場合は識別子を使うことを強く推奨する。

この識別子の様式は Stacks project[*1]や Kerodon[*2]のものと同等ではあるが、本書では (今のところ) 識別子に恒久的な URL は与えられていない。「識別子一覧」のページから識別子の場所を参照できる。

[*1] https://stacks.math.columbia.edu/
[*2] https://kerodon.net/

第 I 部

基礎 [0006]

第 2 章

型理論 [0001]

ホモトピー型理論は**型理論**の一種であり、型理論は一般になんらかの対象を**構成**するための規則群と構成されたものに関する**定義**を表す規則群によって定められる。この章では、ホモトピー型理論の基礎となる構成規則を導入する。

型理論一般における概念や表記をいくつか導入する。以下の用語は形式的に定義することもできるが、当面は普通の自然言語の意味で解釈して問題ない。同じ種類の対象 α と β が**定義により等しい**ことを $\alpha \equiv \beta$ と書く。構成にはいくつかの**仮定**が置かれることもある。α を仮定 $x_1, \ldots, x_n$ の下での対象、a_i を x_i と同じ種類の対象とするとき、α の構成において各 x_i を a_i に**置き換える**ことができ、その結果得られる対象を $\alpha[x_1 \mapsto a_1, \ldots, x_n \mapsto a_n]$ と書く。どのような種類の対象を構成できるかは型理論によって異なるが、通常は**型**とその**要素**が最も興味のある対象である。各要素には型が割り当てられており、要素 a の型が A であることを明示するときは $a : A$ と書く。

2.1 宇宙 [0009]

要素が型であるような型を**宇宙** (universe) と呼ぶ。本書で考える型理論は「可算個」の非有界な宇宙の列

$$\mathcal{U}(0) : \mathcal{U}(1) : \mathcal{U}(2) : \ldots$$

を持つ。

数学を基礎付ける文脈では $0, 1, 2, \ldots$ が何なのか分からないので、**階数** (level) の概念を導入する。階数は型でも要素でもない別の種類の対象である。

規則 2.1.1 ([000D]).

1. 階数 0 を構成できる。
2. 階数 i に対し、階数 $\mathsf{suc}(i)$ を構成できる。

宇宙に関する規則は次の通りである。

規則 2.1.2 ([000E]). 1. 階数 i に対して、型 $\mathcal{U}(i)$ を構成できる。
2. 階数 i と要素 $A : \mathcal{U}(i)$ に対して、型 $\mathsf{El}(A)$ を構成できる。
3. 階数 i に対して、要素 $\lceil\mathcal{U}\rceil(i) : \mathcal{U}(\mathsf{suc}(i))$ を構成できる。
4. 階数 i に対して、$\mathsf{El}(\lceil\mathcal{U}\rceil(i)) \equiv \mathcal{U}(i)$ と定義される。
5. 階数 i と要素 $A : \mathcal{U}(i)$ に対して、$\mathsf{Lift}(A) : \mathcal{U}(\mathsf{suc}(i))$ を構成できる。
6. 階数 i と要素 $A : \mathcal{U}(i)$ に対して、$\mathsf{El}(\mathsf{Lift}(A)) \equiv A$ と定義される。

記法 2.1.3 ([000F]). 1. $\mathsf{El}(\ )$ はよく省略する。つまり、要素 $A : \mathcal{U}(i)$ そのものを型とみなす。
2. $\mathsf{Lift}(\ )$ はよく省略する。つまり、要素 $A : \mathcal{U}(i)$ は $\mathcal{U}(\mathsf{suc}(i))$ の要素でもあるとみなす。
3. $\mathcal{U}(i)$ と $\lceil\mathcal{U}\rceil(i)$ を表記上区別しない。つまり、$\mathcal{U}(i)$ そのものを $\mathcal{U}(\mathsf{suc}(i))$ の要素とみなす。

本書で考える型理論では規則 2.1.2 の他には形式的な意味での型を構成する規則は与えず、代わりに $\mathcal{U}(i)$ の要素を構成する規則を与える。以降、$\mathcal{U}(i)$ の要素を (階数 i の) **型** (type) と呼ぶ。

2.2 関数型 [000A]

関数は型理論において最も基本的な概念である。

規則 2.2.1 ([000H]). i を階数、$A : \mathcal{U}(i)$ を型、$B : \mathcal{U}(i)$ を仮定 $x : A$ の下での型とする。

1. **関数型** (function type) $\prod_{x:A} B : \mathcal{U}(i)$ を構成できる。$\prod_{x:A} B$ の要素を**関数** (function) と呼ぶ。
2. 仮定 $x : A$ の下での要素 $b : B$ に対し、**ラムダ抽象** (lambda abstraction) $\lambda x.b : \prod_{x:A} B$ を構成できる。
3. 要素 $f : \prod_{x:A} B$ と $a : A$ に対し、**関数適用** (function application) $f(a) : B[x \mapsto a]$ を構成できる。
4. $b : B$ を仮定 $x : A$ の下での要素、$a : A$ を要素とする。このとき、$(\lambda x.b)(a) \equiv b[x \mapsto a]$ と定義される。
5. 要素 $f : \prod_{x:A} B$ に対し、$f \equiv \lambda x.f(x)$ と定義される。

関数型の導入により、仮定 $x : A$ の下での B の要素と $\prod_{x:A} B$ 型の関数は同じように振る舞う。以降は仮定の下での要素の代わりに関数を使う。

定義 2.2.2 ([000G]). i を階数、$A, B : \mathcal{U}(i)$ を型とする。このとき、$A \to B : \mathcal{U}(i)$ を

$\prod_{x:A} B$ と定義する。

記法 2.2.3 ([000J]). 1. $\to$ は右結合の演算子である。例えば、$A \to B \to C$ は $A \to (B \to C)$ と読む。
2. $\lambda x_1. \ldots \lambda x_n.b$ は $\lambda x_1 \ldots x_n.b$ と略記することがある。
3. $f(a_1) \ldots (a_n)$ は $f(a_1, \ldots, a_n)$ と略記することがある。
4. $\prod_{x:A}$ の結合は弱い。例えば、$\prod_{x:A} \prod_{y:B} C \to D$ は $\prod_{x:A}(\prod_{y:B}(C \to D))$ と読む。

用語 2.2.4 ([000I]). i を階数、$A : \mathcal{U}(i)$ を型とする。関数 $B : A \to \mathcal{U}(i)$ を A 上の**型の族** (type family) と呼ぶ。

例 2.2.5 ([0010]). i を階数、$A : \mathcal{U}(i)$ を型とする。**恒等関数** (identity function) $\mathsf{id}(A) : A \to A$ を $\lambda x.x$ と定義する。

例 2.2.6 ([0011]). i を階数、$A, B, C : \mathcal{U}(i)$ を型、$f : A \to B$ と $g : B \to C$ を関数とする。**合成関数** (composed function) $g \circ f : A \to C$ を $\lambda x.g(f(x))$ と定義する。

演習 2.2.7 ([0012]). i を階数、$A, B, C, D : \mathcal{U}(i)$ を型、$f : A \to B$, $g : B \to C$, $h : C \to D$ を関数とする。

1. $f \equiv f \circ \mathsf{id}(A)$ であることを確かめよ。
2. $\mathsf{id}(B) \circ f \equiv f$ であることを確かめよ。
3. $(h \circ g) \circ f \equiv h \circ (g \circ f)$ であることを確かめよ。

演習 2.2.8 ([0013]). i を階数、$A, B : \mathcal{U}(i)$ を型、$C : A \to B \to \mathcal{U}(i)$ を型の族、$f : \prod_{x:A} \prod_{y:B} C(x, y)$ を関数とする。関数 $\mathsf{swap}(f) : \prod_{y:B} \prod_{x:A} C(x, y)$ であって、任意の要素 $a : A$ と $b : B$ に対して $\mathsf{swap}(f)(b, a) \equiv f(a, b)$ となるものを構成せよ。

記法 2.2.9 ([000Q]). $f : \prod_{[x:A]} \prod_{y:B} C$ のように引数を [ ] で囲った場合、その引数は**暗黙的** (implicit) であると約束する。つまり、要素 $a : A$ と $b : B[x \mapsto a]$ に対して、関数適用を $f(a, b)$ の代わりに a を省略して $f(b)$ と書く。B の構成で x を明示的に使う場合には実用上は曖昧性は無い。

2.3 レコード型 [000B]

レコード型は構造を記述するのに便利な型である。本書では、組み込み型としては単位型と対型を導入し、レコード型は記法として実現する。これは体系を単純なものに抑えるためである。

規則 2.3.1 ([000K]). 1. **単位型** (unit type) $\mathbf{1} : \mathcal{U}(0)$ を構成できる。
2. 要素 $\star : \mathbf{1}$ を構成できる。
3. 要素 $a : \mathbf{1}$ に対し、$a \equiv \star$ と定義される。

規則 2.3.2 ([000L]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族とする。

1. **対型** (pair type) $\sum_{x:A} B(x) : \mathcal{U}(i)$ を構成できる。
2. 要素 $a : A$ と $b : B(a)$ に対し、**対** (pair) $\mathsf{pair}(a, b) : \sum_{x:A} B(x)$ を構成できる。
3. 要素 $c : \sum_{x:A} B(x)$ に対し、**射影** (projection) $\mathsf{pr}_1(c) : A$ と $\mathsf{pr}_2(c) : B(\mathsf{pr}_1(c))$ を構成できる。
4. 要素 $a : A$ と $b : B(a)$ に対し、$\mathsf{pr}_1(\mathsf{pair}(a, b)) \equiv a$, $\mathsf{pr}_2(\mathsf{pair}(a, b)) \equiv b$ と定義される。
5. 要素 $c : \sum_{x:A} B(x)$ に対し、$c \equiv \mathsf{pair}(\mathsf{pr}_1(c), \mathsf{pr}_2(c))$ と定義される。

定義 2.3.3 ([000M]). i を階数、$A, B : \mathcal{U}(i)$ を型とする。型 $A \times B : \mathcal{U}(i)$ を $\sum_{x:A} B$ と定義する。

記法 2.3.4 ([000N]). 1. $\times$ は右結合の演算子である。例えば、$A \times B \times C$ は $A \times (B \times C)$ と読む。
2. $\sum_{x:A}$ の結合は弱い。例えば、$\sum_{x:A} \sum_{y:B} C \times D$ は $\sum_{x:A} (\sum_{y:B} (C \times D))$ と読む。

例 2.3.5 ([0014]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$C : \prod_{x:A} B(x) \to \mathcal{U}(i)$ を型の族とする。

1. 関数 $f : \prod_{z:\sum_{x:A} B(x)} C(\mathsf{pr}_1(z), \mathsf{pr}_2(z))$ に対し、**カリー化** (currying) $\mathsf{curry}(f) : \prod_{x:A} \prod_{y:B(x)} C(x, y)$ を $\lambda xy.f(\mathsf{pair}(x, y))$ と定義する。
2. 関数 $g : \prod_{x:A} \prod_{y:B(x)} C(x, y)$ に対し、**逆カリー化** (uncurrying) $\mathsf{uncurry}(g) : \prod_{z:\sum_{x:A} B(x)} C(\mathsf{pr}_1(z), \mathsf{pr}_2(z))$ を $\lambda z.g(\mathsf{pr}_1(z), \mathsf{pr}_2(z))$ と定義する。

演習 2.3.6 ([0015]). 例 2.3.5 において、$\mathsf{uncurry}(\mathsf{curry}(f)) \equiv f$ と $\mathsf{curry}(\mathsf{uncurry}(g)) \equiv g$ を確かめよ。

例 2.3.7 ([0016]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$C : \prod_{x:A} B(x) \to \mathcal{U}(i)$ を型の族とする。

1. 関数 $\mathsf{assoc}(C) : (\sum_{z:\sum_{x:A} B(x)} C(\mathsf{pr}_1(z), \mathsf{pr}_2(z))) \to (\sum_{x:A} \sum_{y:B(x)} C(x, y))$ を $\lambda w.\mathsf{pair}(\mathsf{pr}_1(\mathsf{pr}_1(w)), \mathsf{pair}(\mathsf{pr}_2(\mathsf{pr}_1(w)), \mathsf{pr}_2(w)))$ と定義する。
2. 関数 $\mathsf{assoc}^{-1}(C) : (\sum_{x:A} \sum_{y:B(x)} C(x, y)) \to (\sum_{z:\sum_{x:A} B(x)} C(\mathsf{pr}_1(z), \mathsf{pr}_2(z)))$ を $\lambda w.\mathsf{pair}(\mathsf{pair}(\mathsf{pr}_1(w), \mathsf{pr}_1(\mathsf{pr}_2(w))), \mathsf{pr}_2(\mathsf{pr}_2(w)))$ と定義する。

演習 2.3.8 ([0017]). 例 2.3.7 において、$\mathsf{assoc}^{-1}(C) \circ \mathsf{assoc}(C) \equiv \mathsf{id}(_)$ と $\mathsf{assoc}(C) \circ \mathsf{assoc}^{-1}(C) \equiv \mathsf{id}(_)$ を確かめよ。

例 2.3.9 ([0018]). i を階数、$A, B : \mathcal{U}(i)$ を型とする。関数 $\mathsf{sym}(A, B) : A \times B \to B \times A$ を $\lambda z.\mathsf{pair}(\mathsf{pr}_2(z), \mathsf{pr}_1(z))$ と定義する。

演習 2.3.10 ([0019]). 例 2.3.9 において、$\mathsf{sym}(B, A) \circ \mathsf{sym}(A, B) \equiv \mathsf{id}(_)$ であることを確かめよ。

例 2.3.11 ([001A]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$C : \prod_{x:A} B(x) \to \mathcal{U}(i)$ を型の族とする。

1. 関数 $\mathsf{dist}(C) : (\prod_{x:A} \sum_{y:B(x)} C(x, y)) \to (\sum_{f:\prod_{x:A} B(x)} \prod_{x:A} C(x, f(x)))$ を $\lambda h.\mathsf{pair}(\lambda x.\mathsf{pr}_1(h(x)), \lambda x.\mathsf{pr}_2(h(x)))$ と定義する。
2. 関数 $\mathsf{dist}^{-1}(C) : (\sum_{f:\prod_{x:A} B(x)} \prod_{x:A} C(x, f(x))) \to (\prod_{x:A} \sum_{y:B(x)} C(x, y))$ を $\lambda k.\lambda x.\mathsf{pair}(\mathsf{pr}_1(k)(x), \mathsf{pr}_2(k)(x))$ と定義する。

演習 2.3.12 ([001B]). 例 2.3.11 において、$\mathsf{dist}^{-1}(C) \circ \mathsf{dist}(C) \equiv \mathsf{id}(_)$ と $\mathsf{dist}(C) \circ \mathsf{dist}^{-1}(C) \equiv \mathsf{id}(_)$ を確かめよ。

演習 2.3.13 ([002M]). i を階数、$A : \mathcal{U}(i)$ を型とする。関数 $\mathsf{diag}(A) : A \to A \times A$ であって、任意の $a : A$ に対して $\mathsf{pr}_1(\mathsf{diag}(A)(a)) \equiv a$, $\mathsf{pr}_2(\mathsf{diag}(A)(a)) \equiv a$ となるものを構成せよ。

記法 2.3.14 ([000O]). 記法 $\{\!\!\{x_1 : A_1, \ldots, x_n : A_n\}\!\!\}$ を次のように定める。

- $\{\!\!\{\}\!\!\}$ は $\mathbf{1}$ のこととする。
- $\{\!\!\{x_1 : A_1, \ldots, x_{n+1} : A_{n+1}\}\!\!\}$ は $\sum_{x_1:A_1} \{\!\!\{x_2 : A_2, \ldots, x_{n+1} : A_{n+1}\}\!\!\}$ のこととする。

この形の型を**レコード型** (record type) と呼ぶ。また、記法 $(\!\!(x_1 \equiv a_1, \ldots, x_n \equiv a_n)\!\!)$ を次のように定める。

- $(\!\!()\!\!)$ は $\star$ のこととする。
- $(\!\!(x_1 \equiv a_1, \ldots, x_{n+1} \equiv a_{n+1})\!\!)$ は $\mathsf{pair}(a_1, (\!\!(x_1 \equiv a_1, \ldots, x_{n+1} \equiv a_{n+1})\!\!))$ のこととする。

これらの記法は、各 a_i が $A_i[x_1 \mapsto a_1, \ldots, x_{i-1} \mapsto a_{i-1}]$ の要素の時に $(\!\!(x_1 \equiv a_1, \ldots, x_n \equiv a_n)\!\!)$ が $\{\!\!\{x_1 : A_1, \ldots, x_n : A_n\}\!\!\}$ の要素になるように設計されている。さらに、$a : \{\!\!\{x_1 : A_1, \ldots, x_n : A_n\}\!\!\}$ 要素、y を x_1 から x_n のいずれかとする時、記法 $a.y$ を次のように定める。

- y が x_1 の時、$a.x_1$ は $\mathsf{pr}_1(a)$ のこととする。

- y が x_2 から x_n のいずれかの時、$a.y$ は $(\mathsf{pr}_2(a) : \{\!\!\{x_2 : A_2, \dots, x_n : A_n\}\!\!\}).y$ のこととする。

この記法は $a.x_i : A_i[x_1 \mapsto a.x_1, \dots x_{i-1} \mapsto a.x_{i-1}]$ となるように設計されている。また、$(\!\!(x_1 \equiv a_1, \dots, x_n \equiv a_n)\!\!).x_i \equiv a_i$ であることと $a \equiv (\!\!(x_1 \equiv a.x_1, \dots, x_n \equiv a.x_n)\!\!)$ であることを確かめられる。

記法 2.3.15 ([000W]). 大きなレコード型を定義する際には、文章内で $\{\!\!\{x_1 : A_1, \dots, x_n : A_n\}\!\!\}$ と書く代わりに縦に並べて

- $x_1 : A_1$
- $\vdots$
- $x_n : A_n$

と書くことがある。レコード型の要素を定義する際にも同様に縦に並べて書くことがある。

例 2.3.16 ([001T]). i を階数、$A, B : \mathcal{U}(i)$ を型とする。型 $A \leftrightarrow B : \mathcal{U}(i)$ を $\{\!\!\{\mathsf{to} : A \to B, \mathsf{from} : B \to A\}\!\!\}$ と定義する。$A \leftrightarrow B$ の要素がある時、A と B は**論理的に同値** (logically equivalent) であると言う。**反射律**、**対称律**、**推移律**を次のように構成できる。

- $(\!\!(\mathsf{to} \equiv \mathsf{id}(A), \mathsf{from} \equiv \mathsf{id}(A))\!\!) : A \leftrightarrow A$
- $\lambda e.(\!\!(\mathsf{to} \equiv e.\mathsf{from}, \mathsf{to} \equiv e.\mathsf{to})\!\!) : (A \leftrightarrow B) \to (B \leftrightarrow A)$
- $\lambda ef.(\!\!(\mathsf{to} \equiv f.\mathsf{to} \circ e.\mathsf{to}, \mathsf{from} \equiv e.\mathsf{from} \circ f.\mathsf{from})\!\!) : (A \leftrightarrow B) \to (B \leftrightarrow C) \to (A \leftrightarrow C)$

演習 2.3.17 ([001U]). i を階数、$A_1, A_2, A_3 : \mathcal{U}(i)$ を型とする。関数 $f : A_1 \to A_2$, $g : A_2 \to A_3$, $h : A_3 \to A_1$ がある時、各 A_n と A_m は論理的に同値であることを確かめよ。

2.4 同一視型 [000C]

同一視型はホモトピー型理論において最も特徴的な型である。

規則 2.4.1 ([000P]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1 : A$ を要素とする。

1. 要素 $a_2 : A$ に対し、**同一視型** (identity type) $a_1 = a_2 : \mathcal{U}(i)$ を構成できる。$a_1 = a_2$ の要素を a_1 と a_2 の**同一視** (identification) と呼ぶ。
2. 要素 $\mathsf{refl}(a_1) : a_1 = a_1$ を構成できる。
3. $a_2 : A$ と $p : a_1 = a_2$ を要素、j を階数、$B : \prod_{[x:A]} a_1 = x \to \mathcal{U}(j)$ を型の族、$b : B(\mathsf{refl}(a_1))$ を要素とすると、要素 $\mathsf{ind}_=(p, B, b) : B(p)$ を構成できる。
4. j を階数、$B : \prod_{[x:A]} a_1 = x \to \mathcal{U}(j)$ を型の族、$b : B(\mathsf{refl}(a_1))$ を要素とすると、

$\mathsf{ind}_{=}(\mathsf{refl}(a_1), B, b) \equiv b$ と定義される。

[HoTT Book] にならって同一視型に等号の記号を使うが、その意味は従来の数学の等号とは大きく異なる。従来の数学では、$a_1 = a_2$ といえば a_1 と a_2 が等しいという命題であるが、型理論では $a_1 = a_2$ はあくまで型である。従って、$a_1 = a_2$ の要素というものを考えることができ、それは非形式的には a_1 と a_2 の同一視のしかたと解釈される。

規則 2.4.1 について説明する。$\mathsf{refl}(a_1) : a_1 = a_1$ は a_1 とそれ自身の自明な同一視を表す。残りの規則はいわゆる**帰納法原理**の一例で、任意の $a_2 : A$ と同一視型の要素 $p : a_1 = a_2$ を使ってなんらかを構成するためには p が $\mathsf{refl}(a_1)$ の場合の構成 $(b : B(\mathsf{refl}(a_1)))$ を与えれば十分であると読める。注意するべきこととして、この帰納法原理から $a_1 = a_1$ の要素は $\mathsf{refl}(a_1)$ しか無いことは**導出されない**。同一視型の規則の正しい読み方は型の族 $\lambda x.a_1 = x : A \to \mathcal{U}(i)$ が $\mathsf{refl}(a_1) : a_1 = a_1$ で自由に生成されることであって、個々の型 $a_1 = a_2$ については特に言えることはない。

$a_1 = a_2$ が同一視型と呼ぶに価することを確認するために、いくつかの期待される関数を構成しよう。

定義 2.4.2 ([001C]). i を階数、$A : \mathcal{U}(i)$ を型、j を階数、$B : A \to \mathcal{U}(j)$ を型の族とする。$a_1, a_2 : A$ と $p : a_1 = a_2$ を要素とする。**輸送関数** (transport function) $\mathsf{transport}(B, p) : B(a_1) \to B(a_2)$ を $\lambda y_1.\mathsf{ind}_{=}(p, \lambda xz.B(x), y_1)$ と定義する。$\mathsf{transport}(B, \mathsf{refl}(a_1)) \equiv \mathsf{id}(B(a_1))$ であることが確かめられる。

比較 2.4.3 ([001G]). 述語論理において、B を一変数の述語とすると、$x_1 = x_2 \to B(x_1) \to B(x_2)$ が成り立つ。これは等しい対象は述語によって区別できないことを表す。定義 2.4.2 はこの類似で、型理論においては同一視される要素を型の族では区別できないことを表す。

定義 2.4.4 ([001D]). i を階数、$A : \mathcal{U}(i)$ を型、$a_0, a_1, a_2 : A$ を要素、$p_1 : a_0 = a_1$ と $p_2 : a_0 = a_2$ を同一視とする。同一視 $\mathsf{ext}(p_1, p_2) : a_1 = a_2$ を $\mathsf{transport}(\lambda x.x = a_2, p_1)(p_2)$ と定義する。

定義 2.4.5 ([001E]). i を階数、$A : \mathcal{U}(i)$ を型とする。

1. 関数 $\mathsf{sym} : \prod_{[x_1, x_2 : A]} x_1 = x_2 \to x_2 = x_1$ を $\lambda x_1 x_2 z.\mathsf{ext}(z, \mathsf{refl}(x_1))$ と定義する。$\mathsf{sym}(p)$ を p^{-1} とも書く。
2. 関数 $\mathsf{trans} : \prod_{[x_1, x_2, x_3 : A]} x_1 = x_2 \to x_2 = x_3 \to x_1 = x_3$ を $\lambda x_1 x_2 x_3 zw.\mathsf{ext}(z^{-1}, w)$ と定義する。$\mathsf{trans}(p, q)$ を $q \circ p$ とも書く。

比較 2.4.6 ([001H]). 述語論理において、**対称律** $x_1 = x_2 \to x_2 = x_1$ と**推移律** $x_1 = x_2 \to x_2 = x_3 \to x_1 = x_3$ が成り立つ。定義 2.4.5 はこの類似である。ちなみに、

$\mathsf{refl}(x) : x = x$ は**反射律**とも思える。

定義 2.4.7 ([001F]). i 階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。関数 $\mathsf{ap}(f) : \prod_{[x_1, x_2 : A]} x_1 = x_2 \to f(x_1) = f(x_2)$ を $\lambda x_1 x_2 z.\mathsf{transport}(\lambda x.f(x_1) = f(x), z)(\mathsf{refl}(f(x_1)))$ と定義する。文脈上わかる場合は、$\mathsf{ap}(f)(p)$ のことを $f(p)$ と書いてしまうこともある。

比較 2.4.8 ([001I]). 述語論理において、f を一変数の関数とすると、$x_1 = x_2 \to f(x_1) = f(x_2)$ が成り立つ。これは関数が等しさを保つことを表す。定義 2.4.7 はこの類似で、型理論において関数は同一視を保つことを表す。

2.4.1　高次グルーポイド構造 [002N]

要素 $a_1, a_2 : A$ に対して、$a_1 = a_2$ がまた型であるということは、同一視 $p_1, p_2 : a_1 = a_2$ に対してもまた同一視型 $p_1 = p_2$ がある。同一視 $q_1, q_2 : p_1 = p_2$ に対してさらに同一視型 $q_1 = q_2$ があり、これを繰り返すと好きなだけ「高次の」同一視型を得る。実は、これらの高次の同一視型たちは**∞グルーポイド**と呼ばれる構造の一部であることが知られている [3, 1]。∞グルーポイドの構造は無限個の演算を持つ複雑なもので深入りはしない。ただ覚えておくとよいのは、あらゆる型が自動的にそのような豊富な構造を持ち、あらゆる構成が自動的にその構造と整合的になるということである。

∞グルーポイドの持つ構造の一部を挙げる。∞グルーポイドの文脈では要素 $p : a_1 = a_2$ は**射**と呼ばれる。$\mathsf{refl}(a_1) : a_1 = a_1$ は**恒等射**、$p^{-1} : a_2 = a_1$ は**逆射**、$q \circ p : a_1 = a_3$ は**合成射**と呼ばれる。$\mathsf{ext}(p_1, p_2) : a_1 = a_2$ は $p_2 : a_0 = a_2$ の $p_1 : a_0 = a_1$ に沿った**拡張**である。

例 2.4.9 ([001M]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2 : A$ を要素、$p : a_1 = a_2$ を同一視とする。同一視 $\mathsf{ext\text{-}self}(p) : \mathsf{ext}(p, p) = \mathsf{refl}(a_2)$ を次のように構成する。同一視型の帰納法により、a_2 が a_1 で p が $\mathsf{refl}(a_1)$ の場合を考えれば十分である。定義より $\mathsf{ext}(\mathsf{refl}(a_1), \mathsf{refl}(a_1)) \equiv \mathsf{refl}(a_1)$ なので $\mathsf{ext\text{-}self}(\mathsf{refl}(a_1)) \equiv \mathsf{refl}(\mathsf{refl}(a_1))$ と定義する。形式的に書けば $\mathsf{ext\text{-}self}(p) \equiv \mathsf{ind}_=(p, \lambda xz.\mathsf{ext}(z, z) = \mathsf{refl}(x), \mathsf{refl}(\mathsf{refl}(a_1)))$ である。

例 2.4.10 ([0027]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2 : A$ を要素、$p : a_1 = a_2$ を同一視とする。同一視 $\mathsf{sym\text{-}sym}(p) : (p^{-1})^{-1} = p$ を帰納法により $\mathsf{sym\text{-}sym}(\mathsf{refl}(a_1)) \equiv \mathsf{refl}(\mathsf{refl}(a_1))$ と定義する。

例 2.4.11 ([002O]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2, a_3, a_4 : A$ を要素、$p_1 : a_1 = a_2$ と $p_2 : a_2 = a_3$ と $p_3 : a_3 = a_4$ を同一視とする。同一視 $\mathsf{assoc}(p_3, p_2, p_1) : (p_3 \circ p_2) \circ p_1 = p_3 \circ (p_2 \circ p_1)$ を構成する。同一視型の帰納法により、$\mathsf{assoc}(p_3, p_2, \mathsf{refl}(a_1)) \equiv \mathsf{refl}(p_3 \circ p_2)$ と定義すればよい。

演習 2.4.12 ([002P]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2 : A$ を要素、$p : a_1 = a_2$ を同一視とする。同一視 $\mathsf{unit\text{-}l}(p) : \mathsf{refl}(a_2) \circ p = p$ と $\mathsf{unit\text{-}r}(p) : p = p \circ \mathsf{refl}(a_1)$ を構成せよ。

演習 2.4.13 ([0047]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2 : A$ を要素、$p : a_1 = a_2$ を同一視とする。同一視 $\mathsf{inv\text{-}l}(p) : p^{-1} \circ p = \mathsf{refl}(a_1)$ と $\mathsf{inv\text{-}r}(p) : p \circ p^{-1} = \mathsf{refl}(a_2)$ を構成せよ。

演習 2.4.14 ([002R]). いわゆる *Mac Lane* **五角形**を表す型とその要素を構成せよ。

例 2.4.15 ([002Q]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。

1. 要素 $a_1, a_2, a_3 : A$ と同一視 $p_1 : a_1 = a_2$ と $p_2 : a_2 = a_3$ に対して、同一視 $\mathsf{ap\text{-}comp}(f, p_2, p_1) : \mathsf{ap}(f)(p_2 \circ p_1) = \mathsf{ap}(f)(p_2) \circ \mathsf{ap}(f)(p_1)$ を構成できる。実際、$\mathsf{ap\text{-}comp}(f, p_2, \mathsf{refl}(a_1)) \equiv \mathsf{refl}(\mathsf{ap}(f)(p_2))$ と定義すればよい。
2. 要素 $a_1, a_2 : A$ と同一視 $p : a_1 = a_2$ に対して、同一視 $\mathsf{ap\text{-}inv}(f, p) : \mathsf{ap}(f)(p^{-1}) = \mathsf{ap}(f)(p)^{-1}$ を構成できる。実際、$\mathsf{ap\text{-}inv}(f, \mathsf{refl}(a_1)) \equiv \mathsf{refl}(_)$ と定義すればよい。

例 2.4.16 ([002S]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族とする。要素 $a_1, a_2, a_3 : A$ と同一視 $p_1 : a_1 = a_2$ と $p_2 : a_2 = a_3$ に対して、同一視 $\mathsf{transport\text{-}comp}(B, p_2, p_1) : \mathsf{transport}(B, p_2 \circ p_1) = \mathsf{transport}(B, p_2) \circ \mathsf{transport}(B, p_1)$ を構成できる。実際、$\mathsf{transport\text{-}comp}(B, p_2, \mathsf{refl}(a_1)) \equiv \mathsf{refl}(_)$ と定義すればよい。

2.5 自然数 [002T]

今までに導入した型だけでは何も面白い数学はできない。数学を面白くするには**自然数**の概念は必須であろう。自然数の型は**帰納的型**の例として定義される。

規則 2.5.1 ([002V]).

1. **自然数型** (type of natural numbers) $\mathbb{N} : \mathcal{U}(0)$ を構成できる。$\mathbb{N}$ の要素を**自然数** (natural number) と呼ぶ。
2. 要素 $0 : \mathbb{N}$ を構成できる。
3. 要素 $n : \mathbb{N}$ に対して、要素 $\mathsf{suc}(n) : \mathbb{N}$ を構成できる。
4. $n : \mathbb{N}$ を要素、i を階数、$A : \mathbb{N} \to \mathcal{U}(i)$ を型の族、$a : A(0)$ を要素、$f : \prod_{[x:\mathbb{N}]} A(x) \to A(\mathsf{suc}(x))$ を関数とすると、要素 $\mathsf{ind}_{\mathbb{N}}(n, A, a, f) : A(n)$ を構成できる。
5. i を階数、$A : \mathbb{N} \to \mathcal{U}(i)$ を型の族、$a : A(0)$ を要素、$f : \prod_{[x:\mathbb{N}]} A(x) \to A(\mathsf{suc}(x))$ を関数とすると、$\mathsf{ind}_{\mathbb{N}}(0, A, a, f) \equiv a$ と定義される。
6. $n : \mathbb{N}$ を要素、i を階数、$A : \mathbb{N} \to \mathcal{U}(i)$ を型の族、$a : A(0)$ を要素、$f : \prod_{[x:\mathbb{N}]} A(x) \to A(\mathsf{suc}(x))$ を関数とすると、$\mathsf{ind}_{\mathbb{N}}(\mathsf{suc}(n), A, a, f) \equiv f(\mathsf{ind}_{\mathbb{N}}(n, A, a, f))$ と定義される。

一般に帰納的型はいくつかの**構成子** (constructor) によって定められる。$\mathbb{N}$ の場合、0 と $\mathsf{suc}(\ )$ が構成子である。これらは $\mathbb{N}$ の要素を構成する方法を与える。任意の $\mathbb{N}$ の要素がこれらの構成子のみを使って構成されることを表すために、**帰納法原理** (induction principle) を規則として認める。$\mathsf{ind}_{\mathbb{N}}(\ ,\ ,\ ,\ )$ に関する規則は、自然数 n を使ってなんらかを構成するには、0 の場合の構成 $(a : A(0))$ と $\mathsf{suc}(x)$ の場合の構成 $(f : \prod_{[x:\mathbb{N}]} A(x) \to A(\mathsf{suc}(x)))$ を与えれば十分であることを意味する。つまり、型理論の中の人にとっては自然数とは 0 と $\mathsf{suc}(\ )$ のみを使って構成されたものである。さらに、$\mathsf{suc}(x)$ の場合の構成においては、x の場合の構成 (f の引数 $y : A(x)$) は既に定義されたと仮定してよい。これは**再帰的定義** (recursive definition) を可能にする。

例 2.5.2 ([002W]). $n_1, n_2 : \mathbb{N}$ を要素とする。要素 $\mathsf{plus}(n_1, n_2) : \mathbb{N}$ を構成する。自然数の帰納法を使い、$\mathsf{plus}(0, n_2) \equiv n_2$, $\mathsf{plus}(\mathsf{suc}(n_1), n_2) \equiv \mathsf{suc}(\mathsf{plus}(n_1, n_2))$ と定義する。形式的には $\mathsf{plus}(n_1, n_2) \equiv \mathsf{ind}_{\mathbb{N}}(n_1, \lambda x.\mathbb{N}, n_2, \lambda xy.\mathsf{suc}(y))$ と定義できる。

例 2.5.3 ([002X]). $n : \mathbb{N}$ を要素、$f : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ を関数とする。**原始再帰** (primitive recursion) による関数 $\mathsf{prim\text{-}rec}(n, f) : \mathbb{N} \to \mathbb{N}$ が $\mathsf{prim\text{-}rec}(n, f)(0) \equiv n$ と $\mathsf{prim\text{-}rec}(n, f)(\mathsf{suc}(m)) \equiv f(m, \mathsf{prim\text{-}rec}(n, f)(m))$ で定義される。形式的には $\mathsf{prim\text{-}rec}(n, f) \equiv \lambda m.\mathsf{ind}_{\mathbb{N}}(m, \lambda x.\mathbb{N}, n, \lambda xy.f(x, y))$ である。よって、いわゆる**原始再帰的関数**はすべて構成できる。

演習 2.5.4 ([002Y]). *Ackermann* **関数**は二変数の関数 $\mathsf{ack} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ で、次のように定義される。

- $\mathsf{ack}(0, n) \equiv \mathsf{suc}(n)$
- $\mathsf{ack}(\mathsf{suc}(m), 0) \equiv \mathsf{ack}(m, \mathsf{suc}(0))$
- $\mathsf{ack}(\mathsf{suc}(m), \mathsf{suc}(n)) \equiv \mathsf{ack}(m, \mathsf{ack}(\mathsf{suc}(m), n))$

Ackermann 関数の構成を規則 2.5.1 に基づいて正当化せよ。ちなみに、Ackermann 関数は原始再帰的でないことが知られているので、例 2.5.3 の特別な場合としては定義できない。関数型を使えることに注意するとよい。

2.6 有限余積 [002U]

自然数型の他にも重要な帰納的型がある。最も簡単な帰納的型は**空型**である。

規則 2.6.1 ([0030]).

1. **空型** (empty type) $\mathbf{0} : \mathcal{U}(0)$ を構成できる。
2. $c : \mathbf{0}$ を要素、i を階数、$A : \mathbf{0} \to \mathcal{U}(i)$ を型の族とすると、要素 $\mathsf{ind}_{\mathbf{0}}(c, A) : A(c)$ を構成できる。

空型には構成子が一つも与えられていない。よって、型理論の中の人にとっては $\mathbf{0}$ の要素は存在しないはずである。言い換えれば、$\mathbf{0}$ の要素は**矛盾** (contradiction) を表し、帰納法原理は矛盾からはすべてを導けること、**爆発原理** (principle of explosion または *ex falso quodlibet*) を意味する。

次に簡単な帰納的型は**余積**である。

規則 2.6.2 ([002Z]). i を階数、$A, B : \mathcal{U}(i)$ を型とする。

1. **余積** (coproduct) $A + B : \mathcal{U}(i)$ を構成できる。
2. 要素 $a : A$ に対して、要素 $\mathsf{in}_1(a) : A + B$ を構成できる。
3. 要素 $b : B$ に対して、要素 $\mathsf{in}_2(b) : A + B$ を構成できる。
4. $c : A + B$ を要素、j を階数、$D : A + B \to \mathcal{U}(j)$ を型の族、$d_1 : \prod_{x:A} D(\mathsf{in}_1(x))$ を要素、$d_2 : \prod_{y:B} D(\mathsf{in}_2(y))$ を要素とすると、要素 $\mathsf{ind}_+(c, D, d_1, d_2) : D(c)$ を構成できる。
5. $a : A$ を要素、j を階数、$D : A + B \to \mathcal{U}(j)$ を型の族、$d_1 : \prod_{x:A} D(\mathsf{in}_1(x))$ を要素、$d_2 : \prod_{y:B} D(\mathsf{in}_2(y))$ を要素とすると、$\mathsf{ind}_+(\mathsf{in}_1(a), D, d_1, d_2) \equiv d_1(a)$ と定義される。
6. $b : B$ を要素、j を階数、$D : A + B \to \mathcal{U}(j)$ を型の族、$d_1 : \prod_{x:A} D(\mathsf{in}_1(x))$ を要素、$d_2 : \prod_{y:B} D(\mathsf{in}_2(y))$ を要素とすると、$\mathsf{ind}_+(\mathsf{in}_2(b), D, d_1, d_2) \equiv d_2(b)$ と定義される。

$\mathsf{in}_1(\ )$ と $\mathsf{in}_2(\ )$ が $A + B$ の構成子であり、帰納法原理はやはりこれらの構成子のみを使って得られるものだけが $A + B$ の要素であることを表す。

第 3 章

一価性公理 [0002]

一価性公理 (univalence axiom) はホモトピー型理論において最も重要な公理で、型理論をまさに「ホモトピー論的」なものに強制する公理である。非形式的には、型 $A, B : \mathcal{U}(i)$ に対して同一視型 $A = B$ と**同値**のなす型 $A \simeq B$ が同値になることを要請する。

一価性について述べるには同値の概念が不可欠である。同値の定義はいくつか考えられるが、本書では関数であって各点の逆像が**可縮**であるものという定義を採用する。型が可縮であるとは非形式的には要素をただ一つ持つということであり、この同値の定義は直観的にも自然であろう。技術的な利点として、可縮性は非常に扱いやすい性質であるという点、ホモトピー型理論における他の概念も可縮性を軸に定義することができるという点がある。3.1 節で可縮性についての基本事項まとめる。

一価性公理は宇宙 $\mathcal{U}(i)$ の同一視型の特徴付けを与えているとも解釈できる。諸々の型の同一視型の特徴付けについての一般論として、**同一視型の基本定理**を 3.2 節で証明する。「基本定理」と名付けられる通り、この定理は本書の至る所で使われる。3.3 節で一価性公理を導入する。また、関数型の同一視型の特徴付け (いわゆる**関数外延性**) にもなんらかの公理が必要であることが知られているので、3.4 節で**関数外延性公理**を導入する。

一価性公理は後に導入する高次帰納的型と合わせてその真価を発揮するのだが、一価性公理だけでもいくつか興味深い帰結を得られる。その一つは**構造同一原理** (structure identity principle) と呼ばれるもので、なんらかの数学的構造の同一視型はその構造を保つ同値の型と同値であるという原理である。構造同一原理を一般的に述べるには「構造」とは厳密にどう定義されるかという問題が大きいので立ち入らず、3.5 節でいくつかの例を挙げるにとどめる。

3.6 節はやや技術的な話で、同値の概念の基本性質を見る。

3.1 可縮性 [000S]

可縮性はホモトピー型理論において中心的な役割を果たす概念である。

定義 3.1.1 ([000T]). i を階数、$A : \mathcal{U}(i)$ を型とする。型 $\mathsf{IsContr}(A) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{center} : A$
- $\mathsf{contr} : \prod_{x:A} \mathsf{center} = x$

$\mathsf{IsContr}(A)$ の要素がある時、A は**可縮** (contractible) であると言う。

つまり、型 A が可縮であるとは、ある要素 $\mathsf{center} : A$ があり、すべての A の要素は center と同一視されるということである。やや非形式的に言うと、A はただ一つの要素を持つということである。

後に示すことだが、$\mathsf{IsContr}(A)$ の任意の二つの要素は (関数外延性の下で) 同一視される (命題 5.1.6)。したがって、A が可縮であると言った場合、$\mathsf{IsContr}(A)$ の要素があることは重要だが、その要素の具体的な定義は気にしなくてよい。あるいは、可縮性は型についての**命題**であり、それを**証明**できることは重要だがどのように証明されたかは重要ではないとも言える。

例 3.1.2 ([001O]). $\mathbf{1}$ は可縮である。実際、$(\!|\mathsf{center} \equiv \star, \mathsf{contr} \equiv \lambda x.\mathsf{refl}(\star)|\!) : \mathsf{IsContr}(\mathbf{1})$ を確かめられる。

型は∞グルーポイドの構造を持っているので、ただ一つの要素を持つとはいっても異なる同一視のしかたがある可能性が懸念される。次の命題 3.1.3 は、可縮性は見た目よりもずっと強い条件で、任意の二つの要素の同一視もただ一つだけあることを示す。命題 3.1.3 を繰り返し使えば、任意に高次の同一視もただ一つだけあることが分かる。

命題 3.1.3 ([001L]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2 : A$ を要素とする。A が可縮ならば $a_1 = a_2$ も可縮である。

証明. $c : \mathsf{IsContr}(A)$ と仮定する。要素 $p : a_1 = a_2$ と $q : \prod_{z:a_1=a_2} p = z$ を構成すればよい。仮定 c より $a_0 : A, r : \prod_{x:A} a_0 = x$ を得る。$p \equiv \mathsf{ext}(r(a_1), r(a_2))$ と定義する。q については、一般化したもの $q' : \prod_{x:A} \prod_{z:a_1=x} \mathsf{ext}(r(a_1), r(x)) = z$ を構成し、$q \equiv q'(a_2)$ と定義する。同一視型の帰納法により、要素 $s : \mathsf{ext}(r(a_1), r(a_1)) = \mathsf{refl}(a_1)$ を構成すればよいが、これは例 2.4.9 で構成した。 □

型の可縮性は次の命題 3.1.4 や命題 3.1.6 を使って示される場合が多い。

命題 3.1.4 ([001N]). i を階数、$A : \mathcal{U}(i)$ を型、$a : A$ を要素とする。型 $\sum_{x:A} a = x$ は可縮である。

証明. 要素 $c : \sum_{x:A} a = x$ と $r : \prod_{z:\sum_{x:A} a=x} c = z$ を構成すればよい。$c \equiv \mathsf{pair}(a, \mathsf{refl}(a))$ と定義する。r については、カリー化 (例 2.3.5) により、

$r' : \prod_{x:A}\prod_{w:a=x} c = \mathsf{pair}(x,w)$ を構成すればよい。同一視型の帰納法により、$c = \mathsf{pair}(a, \mathsf{refl}(a))$ の要素を構成すればよいが、c の定義より $\mathsf{refl}(c)$ とすればよい。 □

命題 3.1.4 は同一視型の帰納法の別表現と考えられる。同一視型の帰納法は型の族 $\lambda x.a = x : A \to \mathcal{U}(i)$ が $\mathsf{refl}(a) : a = a$ で自由に生成されることを表す。これは、対型 $\sum_{x:A} a = x$ が $\mathsf{pair}(a, \mathsf{refl}(a))$ で自由に生成されると言い換えられる。一つの要素で生成される型はその要素しか持たないと期待され、命題 3.1.4 は実際にそうだと言っている。

命題 3.1.6 は可縮性が**レトラクト**で閉じることを主張する。

定義 3.1.5 ([001J]). i を階数、$A, B : \mathcal{U}(i)$ を型とする。型 $\mathsf{Retract}(A,B) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{retraction} : B \to A$
- $\mathsf{section} : A \to B$
- $\mathsf{r\text{-}s} : \prod_{x:A} \mathsf{retraction}(\mathsf{section}(x)) = x$

$\mathsf{Retract}(A,B)$ は $A \lhd B$ と書くこともある。$A \lhd B$ の要素がある時、A は B の**レトラクト** (retract) であると言う。また、$A \lhd\rhd B$ を $(A \lhd B) \times (B \lhd A)$ と定義する。

$\mathsf{Retract}(A,B)$ は $\mathsf{IsContr}(A)$ と違って命題ではないが、実用上は $\mathsf{Retract}(A,B)$ の要素の具体的な定義は重要ではない場合が多い。これは命題 3.1.6 のように、ある性質がレトラクトで閉じるという使われ方をする場合が多いからである。

命題 3.1.6 ([001K]). i を階数、$A, B : \mathcal{U}(i)$ を型、$r : \mathsf{Retract}(A,B)$ を要素とする。B が可縮ならば A も可縮である。

証明. $d : \mathsf{IsContr}(B)$ と仮定する。要素 $a : A$ と $p : \prod_{x:A} a = x$ を構成すればよい。仮定 d から $b : B$, $q : \prod_{y:B} b = y$ を得る。仮定 r から $f : B \to A$, $g : A \to B$, $t : \prod_{x:A} f(g(x)) = x$ を得る。$a \equiv f(b) : A$ と定義する。p を定義するために、$x : A$ を仮定する。$q(g(x)) : b = g(x)$, $t(x) : f(g(x)) = x$ に注意して、$p(x) \equiv t(x) \circ \mathsf{ap}(f)(q(g(x)))$ と定義すればよい。 □

可縮性を使って型の**同値**が定義される。

定義 3.1.7 ([001P]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数、$b : B$ を要素とする。型 $\mathsf{Fiber}(f,b) : \mathcal{U}(i)$ を $\{\!|\mathsf{elem} : A, \mathsf{id} : f(\mathsf{elem}) = b|\!\}$ と定義する。

定義 3.1.8 ([001Q]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。型 $\mathsf{IsEquiv}(f) : \mathcal{U}(i)$ を $\prod_{y:B} \mathsf{IsContr}(\mathsf{Fiber}(f,y))$ と定義する。$\mathsf{IsEquiv}(f)$ の要素がある時、f は**同値** (equivalence) であると言う。

定義 3.1.9 ([000V]). i を階数、$A, B : \mathcal{U}(i)$ を型とする。型 $A \simeq B : \mathcal{U}(i)$ を $\{\!\{\mathsf{fun} : A \to B, \mathsf{is\text{-}equiv} : \mathsf{IsEquiv}(\mathsf{fun})\}\!\}$ と定義する。

この定義による同値の概念が妥当なものであるかは自明ではない。直観的には、$\mathsf{Fiber}(f, b)$ は b の f による逆像であり、$\mathsf{IsContr}(\mathsf{Fiber}(f, b))$ は逆像がただ一つの要素を持つことを表す。これはもっともらしい定義だが、$\simeq$ が反射的、対称、推移的であることすら定義 3.1.9 から直ちに分かることではない。3.6 節でこの同値の概念が妥当であることを説明するが、その前にいくつか重要な定理と概念を導入する。

3.2 同一視型の基本定理 [001R]

同一視型はすべての型に対して一様に定義されているが、個々の型については具体的な同一視のしかたが期待される。例えば、対型の要素 $c_1, c_2 : A \times B$ の「自然な」同一視のしかたは $\mathsf{pr}_1(c_1)$ と $\mathsf{pr}_1(c_2)$ を同一視しかつ $\mathsf{pr}_2(c_1)$ と $\mathsf{pr}_2(c_2)$ を同一視することである。つまり、同値 $(c_1 = c_2) \simeq ((\mathsf{pr}_1(c_1) = \mathsf{pr}_1(c_2)) \times (\mathsf{pr}_2(c_1) = \mathsf{pr}_2(c_2)))$ を構成できると期待される (例 3.2.9)。**同一視型の基本定理** (定理 3.2.4) は、この手の同値を構成する手順を与える。

型 $A : \mathcal{U}(i)$ に興味があるとして、要素 $a : A$ に対して同一視型の族 $\lambda x.a = x : A \to \mathcal{U}(i)$ を特徴付けることを考える。具体的に特徴付けの候補 $B : A \to \mathcal{U}(i)$ を見つけたとしよう。これが正しいものなら、$\mathsf{refl}(a)$ に対応する要素 $b : B(a)$ があるはずである。同一視型の帰納法により、b は関数 $b' : \prod_{x:A} a = x \to B(x)$ に拡張される。$b'(x) : a = x \to B(x)$ は「標準的」な比較関数であり、これが同値であることを示したい。$b'(x)$ の同値性を定義に従って示すことは難しくはないが筋が良いとも言えない。同一視型の基本定理は、すべての $b'(x)$ が同値であることと $\sum_{x:A} B(x)$ が可縮であることが論理的に同値であると主張する。次の点から後者の方が示しやすい性質であると思われる。

- 可縮性は様々な型の構成について閉じる (例えば命題 3.1.3 や命題 3.1.4)。
- $\sum_{x:A} B(x)$ の可縮性は a, b, b' に依らない性質である。

補題 3.2.1 ([001V]). i を階数、$A, B : \mathcal{U}(i)$ を型、$e : A \simeq B$ を同値とすると、$\mathsf{Retract}(B, A)$ の要素を構成できる。

証明. 仮定 e から $f : A \to B$ と $H : \mathsf{IsEquiv}(f)$ を得る。任意の $y : B$ に対して $H(y) : \mathsf{IsContr}(\mathsf{Fiber}(f, y))$ を得るので、特に関数 $G : \prod_{y:B} \mathsf{Fiber}(f, y)$ を得る。例 2.3.11 の要領で G から関数 $g : B \to A$ と同一視 $p : \prod_{y:B} f(g(y)) = y$ を得る。これで要素 $(\!(\mathsf{retraction} \equiv f, \mathsf{section} \equiv g, \mathsf{r\text{-}s} \equiv p)\!) : \mathsf{Retract}(B, A)$ を構成できた。 □

補題 3.2.2 ([001X]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$c_1, c_2 : \sum_{x:A} B(x)$ を要素とすると、$\mathsf{Retract}(\sum_{z:\mathsf{pr}_1(c_1)=\mathsf{pr}_1(c_2)} \mathsf{transport}(B, z)(\mathsf{pr}_2(c_1)) =$

$\mathsf{pr}_2(c_2), c_1 = c_2)$ の要素を構成できる。

証明. 関数 $f : (\sum_{z:\mathsf{pr}_1(c_1)=\mathsf{pr}_1(c_2)} \mathsf{transport}(B,z)(\mathsf{pr}_2(c_1)) = \mathsf{pr}_2(c_2)) \to c_1 = c_2$ と $g : c_1 = c_2 \to (\sum_{z:\mathsf{pr}_1(c_1)=\mathsf{pr}_1(c_2)} \mathsf{transport}(B,z)(\mathsf{pr}_2(c_1)) = \mathsf{pr}_2(c_2))$ と同一視 $p : \prod_w g(f(w)) = w$ を構成する。f についてはカリー化、一般化して $f' : \prod_{[x:A]} \prod_{[y:B(x)]} \prod_{z:\mathsf{pr}_1(c_1)=x} \mathsf{transport}(B,z)(\mathsf{pr}_2(c_1)) = y \to c_1 = \mathsf{pair}(x,y)$ を構成すればよいが、同一視型の帰納法により $f'(\mathsf{refl}(\mathsf{pr}_1(c_1)), \mathsf{refl}(\mathsf{pr}_2(c_1))) \equiv \mathsf{refl}(c_1)$ と定義できる。g は一般化して $g' : \prod_{[x:\sum_{x:A} B(x)]} c_1 = x \to (\sum_{z:\mathsf{pr}_1(c_1)=\mathsf{pr}_1(x)} \mathsf{transport}(B,z)(\mathsf{pr}_2(c_1)) = \mathsf{pr}_2(x))$ を帰納法で $g'(\mathsf{refl}(c_1)) \equiv \mathsf{pair}(\mathsf{refl}(\mathsf{pr}_1(c_1)), \mathsf{refl}(\mathsf{pr}_2(c_2)))$ と定義する。p も f と同様にカリー化、一般化して $p' : \prod_{[x]} \prod_{[y]} \prod_{z:\mathsf{pr}_1(c_1)=x} \prod_{w:\mathsf{transport}(B,z)(\mathsf{pr}_2(c_1))=y} g'(f'(z,w)) = \mathsf{pair}(z,w)$ を帰納法により $p'(\mathsf{refl}(\mathsf{pr}_1(c_1)), \mathsf{refl}(\mathsf{pr}_2(c_2))) \equiv \mathsf{refl}(_)$ と定義する。 □

補題 3.2.3 ([001W]). i を階数、$A : \mathcal{U}(i)$ を型、$B, C : A \to \mathcal{U}(i)$ を型の族、$r : \prod_{x:A} \mathsf{Retract}(B(x), C(x))$ を要素とすると、$\mathsf{Retract}(\sum_{x:A} B(x), \sum_{x:A} C(x))$ の要素を構成できる。

証明. 仮定 r から関数 $f : \prod_{[x:A]} B(x) \to C(x)$ と $g : \prod_{[x:A]} C(x) \to B(x)$ と同一視 $p : \prod_{x:A} \prod_{y:B(x)} g(f(y)) = y$ を得る。関数 $F : (\sum_{x:A} B(x)) \to (\sum_{x:A} C(x))$ を $\lambda z.\mathsf{pair}(\mathsf{pr}_1(z), f(\mathsf{pr}_2(z)))$ と定義する。関数 $G : (\sum_{x:A} C(x)) \to (\sum_{x:A} B(x))$ も同様に g を使って定義される。同一視 $P : \prod_{z:\sum_{x:A} B(x)} G(F(z)) = z$ を定義するために、$z : \sum_{x:A} B(x)$ を仮定する。構成から $\mathsf{pr}_1(G(F(z))) \equiv \mathsf{pr}_1(z)$ であり、$p(\mathsf{pr}_1(z), \mathsf{pr}_2(z)) : \mathsf{pr}_2(G(F(z))) = \mathsf{pr}_2(z)$ を得る。補題 3.2.2 を使って、$\mathsf{pair}(\mathsf{refl}(\mathsf{pr}_1(z)), p(\mathsf{pr}_1(z), \mathsf{pr}_2(z)))$ から $G(F(z)) = z$ の要素を構成できる。 □

定理 3.2.4 (**同一視型の基本定理** [001S]). i を階数、$A : \mathcal{U}(i)$ を型、$a : A$ を要素、$B : A \to \mathcal{U}(i)$ を型の族、$b : B(a)$ を要素とする。次の型は論理的に同値である。

1. $\prod_{x:A} \mathsf{IsEquiv}(\lambda(p : a = x).\mathsf{transport}(B,p)(b))$
2. $\prod_{x:A} (a = x) \simeq B(x)$
3. $\prod_{x:A} \mathsf{Retract}(B(x), a = x)$
4. $\mathsf{IsContr}(\sum_{x:A} B(x))$

証明. $\lambda(p : a = x).\mathsf{transport}(B,p)(b)$ の型が $a = x \to B(x)$ であることから、1 から 2 は $\simeq$ の定義から自明である。

2 から 3 は補題 3.2.1 による。

3 から 4 を示す。3 を仮定すると、補題 3.2.3 から $\mathsf{Retract}(\sum_{x:A} B(x), \sum_{x:A} a = x)$ の要素を得る。すると、命題 3.1.4 と命題 3.1.6 より $\sum_{x:A} B(x)$ は可縮である。

最後に 4 から 1 を示す。4 を仮定し、$x : A$ と $y : B(x)$ を仮定する。

$\mathsf{Fiber}(\lambda p.\mathsf{transport}(B,p)(b),y)$ が可縮であることを示す。$\mathsf{Fiber}(_,_)$ の定義より、$\sum_{p:a=x}\mathsf{transport}(B,p)(b)=y$ が可縮であることを示せばよい。補題 3.2.2 と命題 3.1.6 から、$\mathsf{pair}(a,b)=\mathsf{pair}(x,y)$ が可縮であることを示せばよいが、これは仮定と命題 3.1.3 から従う。 □

副産物として、対の同一視型の特徴付けは既に得られている。

系 3.2.5 ([002B]). i を階数、$A:\mathcal{U}(i)$ を型、$B:A\to\mathcal{U}(i)$ を型の族、$c_1,c_2:\sum_{x:A}B(x)$ を要素とすると、同値

$$(c_1=c_2)\simeq\sum_{z:\mathsf{pr}_1(c_1)=\mathsf{pr}_1(c_2)}\mathsf{transport}(B,z)(\mathsf{pr}_2(c_1))=\mathsf{pr}_2(c_2)$$

を構成できる。

証明. 定理 3.2.4 と補題 3.2.2 による。 □

定理 3.2.4 を適用する際に便利な補題を用意する。

補題 3.2.6 ([0024]). i を階数、$A:\mathcal{U}(i)$ を型、$B:A\to\mathcal{U}(i)$ を型の族、$a:A$ を要素とする。A が可縮ならば $(\sum_{x:A}B(x))\lhd\rhd B(a)$ の要素を構成できる。

証明. A が可縮であると仮定する。命題 3.1.3 より、関数 $p:\prod_{x:A}x=a$ を得る。関数 $f:(\sum_{x:A}B(x))\to B(a)$ を $\lambda z.\mathsf{transport}(B,p(\mathsf{pr}_1(z)))(\mathsf{pr}_2(z))$ と定義し、関数 $g:B(a)\to(\sum_{x:A}B(x))$ を $\lambda y.\mathsf{pair}(a,y)$ と定義する。定義より、任意の $z:\sum_{x:A}B(x)$ に対して、$p(\mathsf{pr}_1(z)):\mathsf{pr}_1(z)=\mathsf{pr}_1(g(f(z)))$ と $\mathsf{refl}(_):\mathsf{transport}(B,p(\mathsf{pr}_1(z)))(\mathsf{pr}_2(z))=\mathsf{pr}_2(g(f(z)))$ を得るので、補題 3.2.2 より同一視 $q:\prod_{z:_}g(f(z))=z$ を得る。また、命題 3.1.3 より同一視 $r:p(a)=\mathsf{refl}(a)$ も得られるので、同一視 $\lambda y.\mathsf{ap}(\lambda w.\mathsf{transport}(B,w)(y))(r):\prod_{y:B(a)}f(g(y))=y$ を得る。 □

補題 3.2.7 ([0025]). i を階数、$A:\mathcal{U}(i)$ を型、$B:A\to\mathcal{U}(i)$ を型の族、$C:\prod_{x:A}B(x)\to\mathcal{U}(i)$ を型の族、$a:A$ と $b:B(a)$ を要素とする。$\sum_{x:A}B(x)$ が可縮ならば $(\sum_{x:A}\sum_{y:B(x)}C(x,y))\lhd\rhd C(a,b)$ の要素を構成できる。

証明. 補題 3.2.6 からすぐに従う。 □

例 3.2.8 ([002C]). $a_1,a_2:\mathbf{1}$ を要素とする。同値 $(a_1=a_2)\simeq\mathbf{1}$ を構成しよう。定理 3.2.4 を適用する。$B:\mathbf{1}\to\mathcal{U}(_)$ を $\lambda x.\mathbf{1}$ と定義する。要素 $\star:B(a_1)$ を得る。例 3.1.2 と補題 3.2.6 により、$\sum_{x:\mathbf{1}}B(x)$ は $B(a_1)$ のレトラクトである。再び例 3.1.2 により $B(a_1)$ は可縮なので、命題 3.1.6 により $\sum_{x:\mathbf{1}}B(x)$ は可縮である。

例 3.2.9 ([002D]). i を階数、$A,B:\mathcal{U}(i)$ を型、$c_1,c_2:A\times B$ を要素とする。同値

$(c_1 = c_2) \simeq ((\mathsf{pr}_1(c_1) = \mathsf{pr}_1(c_2)) \times (\mathsf{pr}_2(c_1) = \mathsf{pr}_2(c_2)))$ を構成しよう。定理 3.2.4 を適用する。$E : (A \times B) \to \mathcal{U}(i)$ を $\lambda z.(\mathsf{pr}_1(c_1) = \mathsf{pr}_1(z)) \times (\mathsf{pr}_2(c_1) = \mathsf{pr}_2(z))$ と定義する。要素 $\mathsf{pair}(\mathsf{refl}(_), \mathsf{refl}(_)) : E(c_1)$ を得る。レトラクトの列

$$
\begin{array}{ll}
 & \sum_{z:A\times B} E(z) \\
\lhd & \quad \{\text{並び換え}\} \\
 & \sum_{x:A} \sum_{p:\mathsf{pr}_1(c_1)=x} \sum_{y:B} \mathsf{pr}_2(c_1) = y \\
\lhd & \quad \{\text{命題 3.1.4 と補題 3.2.7}\} \\
 & \sum_{y:B} \mathsf{pr}_2(c_1) = y
\end{array}
$$

を得て、最後の型は命題 3.1.4 により可縮なので、命題 3.1.6 より $\sum_{z:A\times B} E(z)$ も可縮である。

3.3 一価性 [000X]

同値 $A \simeq B$ は型 $A, B : \mathcal{U}(i)$ の「自然な」同一視のしかたと考えられる。つまり、同値 $(A = B) \simeq (A \simeq B)$ が期待される。しかし、関数 $(A \simeq B) \to (A = B)$ は宇宙、関数型、対型、同一視型の規則からは導出できない。実際、型を従来の意味での集合と解釈するモデルを考えると、$A \simeq B$ は A から B への全単射のなす集合と解釈される一方、$A = B$ は A と B が等しい時に限り (一つだけ) 要素を持つような集合と解釈される。異なる集合の間にも全単射は存在する場合があり、その時には関数 $(A \simeq B) \to (A = B)$ は存在しない。

一価性公理は同値 $(A = B) \simeq (A \simeq B)$ を導出する公理である。先に説明したように、この同値は型理論の集合論的解釈とは相反するものである。一価性公理の下では、型は空間のホモトピー型のように振る舞う。その意味で、一価性公理は型理論をホモトピー論的なものに強制する公理と言える。

補題 3.3.1 ([0026]). i を階数、$A : \mathcal{U}(i)$ を型とすると、関数 $\mathsf{id}(A) : A \to A$ は同値である。

証明. $a : A$ を要素とする。$\mathsf{Fiber}(\mathsf{id}(A), a)$ の定義から、$\sum_{x:A} x = a$ が可縮であることを示せばよい。例 2.4.10 より $x = a$ は $a = x$ のレトラクトなので、定理 3.2.4 より $\sum_{x:A} x = a$ は可縮である。 □

定義 3.3.2 ([000Y]). i を階層とする。宇宙 $\mathcal{U}(i)$ が**一価性** (univalence) を満たすとは、$\prod_{X:\mathcal{U}(i)} \mathsf{IsContr}(\sum_{Y:\mathcal{U}(i)} X \simeq Y)$ の要素があることである。

補題 3.3.1 から定理 3.2.4 を適用できて、$\mathcal{U}(i)$ が一価性を満たす時、任意の型 $A, B : \mathcal{U}(i)$ に対して同値 $(A = B) \simeq (A \simeq B)$ を構成できる。

公理 3.3.3 (一価性公理 [000Z])**.** **一価性公理** (univalence axiom) は任意の階数の宇宙が一価性を満たすことを要請する。

ここで公理という言葉を使ったが、規則と公理に本質的な違いは無い。つまり、公理 3.3.3 は任意の階数 i に対して要素 $\mathsf{ua}(i) : \prod_{X:\mathcal{U}(i)} \mathsf{IsContr}(\sum_{Y:\mathcal{U}(i)} X \simeq Y)$ を構成できるという規則だとも言える。本書では、次のように感覚的に使い分ける。

1. 規則は新しい型の種類を導入する際に使われ、その型が何なのかを説明するものである。例えば規則 2.2.1 は関数型を説明するための規則だが、$\mathsf{ua}(i)$ は複合的な型の要素でしかなく、新しい型を説明するものではない。
2. 規則は導入した後はいつでも暗黙的に使うが、公理を使用する際は明示する。これは公理がどのように使われるかを強調するためというのもあるが、相反する公理を導入する可能性も考えると公理を大域的に仮定するのは良くないというのがある。あるいは、規則は型理論の基盤にあたるものだが公理は選択的な機能だと言ってもよい。

ちなみに、公理はある型が可縮であるという形で述べられることが多い。よって、$\mathsf{ua}(i)$ のような名前を付ける必要はない。

3.4　関数外延性 [001Y]

関数 $f, g : \prod_{x:A} B(x)$ の「自然な」同一視のしかたは任意の $x : A$ に対して $f(x)$ と $g(x)$ を同一視することである。つまり、同値 $(f = g) \simeq \prod_{x:A} f(x) = g(x)$ が期待される。しかし、関数 $(\prod_{x:A} f(x) = g(x)) \to (f = g)$ は関数型の規則と同一視型の規則からは構成できないことが知られている [4]。そのため、この同値を得るためには何らかの公理が必要である。

定義 3.4.1 ([001Z])**.** i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族とする。関数型 $\prod_{x:A} B(x)$ が**関数外延性** (function extensionality) を満たすとは、$\prod_{f:\prod_{x:A} B(x)} \mathsf{IsContr}(\sum_{g:\prod_{x:A} B(x)} \prod_{x:A} f(x) = g(x))$ の要素があることである。

要素 $\lambda x.\mathsf{refl}(f(x)) : \prod_{x:A} f(x) = f(x)$ があるので定理 3.2.4 を適用できて、$\prod_{x:A} B(x)$ が関数外延性を持つ時、任意の関数 $f, g : \prod_{x:A} B(x)$ に対して同値 $(f = g) \simeq \prod_{x:A} f(x) = g(x)$ を得る。

公理 3.4.2 (関数外延性公理 [0020])**.** **関数外延性公理** (function extensionality axiom) はすべての関数型が関数外延性を満たすことを要請する。

命題 3.4.3 ([0029])**.** i を階数とする。次は論理的に同値である。

1. $\mathcal{U}(i)$ のすべての関数型が関数外延性を満たす。
2. 任意の型 $A : \mathcal{U}(i)$ と型の族 $B : A \to \mathcal{U}(i)$ に対して、$\prod_{x:A} \mathsf{IsContr}(B(x)) \to \mathsf{IsContr}(\prod_{x:A} B(x))$ の要素がある。

証明. 1 から 2 を示す。$c : \prod_{x:A} \mathsf{IsContr}(B(x))$ を仮定する。仮定 c より、関数 $f : \prod_{x:A} B(x)$ と同一視 $p : \prod_{x:A} \prod_{y:B(x)} f(x) = y$ を得る。同一視 $q : \prod_{g:\prod_{x:A} B(x)} f = g$ を構成すればよい。任意の $g : \prod_{x:A} B(x)$ に対して、$\lambda x.p(x, g(x)) : \prod_{x:A} f(x) = g(x)$ を得るが、関数外延性と定理 3.2.4 により $f = g$ の要素を得る。

2 から 1 を示す。$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$f : \prod_{x:A} B(x)$ を関数とする。例 2.3.11 より、$\sum_{g:\prod_{x:A} B(x)} \prod_{x:A} f(x) = g(x)$ は $\prod_{x:A} \sum_{y:B(x)} f(x) = y$ のレトラクトであるが、後者は仮定と命題 3.1.4 により可縮である。よって、命題 3.1.6 から前者も可縮である。 □

3.4.1 一価性から関数外延性を導く [0021]

実は、一価性公理から関数外延性公理が導出されることが知られている。

3.5 構造同一原理 [0022]

一価性 (と関数外延性) の帰結として、**構造同一原理** (structure identity principle) をいくつか例示する。構造とは厳密に定義はしないが、なんらかの数学的構造を表すレコード型のこととする。構造同一原理は、構造の同一視型は構造を保つ同値の型と同値であることを主張する。

ある構造 A と要素 $a : A$ に対し、同一視型 $\lambda x.a = x$ を特徴付けるには、候補 $E : A \to \mathcal{U}(i)$ を見つけて $\sum_{x:A} E(x)$ が可縮であることを示す。要素 $e : E(a)$ を自然に見つけられる場合がほとんどで、定理 3.2.4 から $(a = x) \simeq E(x)$ を得る。基本的な戦略は目的の型から始めて、既に可縮と分かっている型になるまでレトラクトの列を作ることである (実際には同値の列を作れる場合が多い)。すると命題 3.1.6 により目的の型が可縮であることが従う。レトラクトの列を作る際には補題 3.2.7 が便利で、大きなレコード型の一部に同一視型の特徴付けを知っている部分があれば、そのレコード型はより単純な型のレトラクトであることを示せる。また、レコード型の要素を並び換える関数はレトラクションであることも便利である。

例 3.5.1 ([0023]). i を階数とする。型 $\mathcal{U}_\bullet(i) : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- $\mathsf{Carrier} : \mathcal{U}(i)$
- $\mathsf{point} : \mathsf{Carrier}$

$\mathcal{U}_\bullet(i)$ の要素を (階数 i の) **点付き型** (pointed type) と呼ぶ。$A : \mathcal{U}_\bullet(i)$ に対し、$E : \mathcal{U}_\bullet(i) \to \mathcal{U}(i)$ を $\lambda Z.\sum_{e:A.\mathsf{Carrier}\simeq Z.\mathsf{Carrier}} e(A.\mathsf{point}) = Z.\mathsf{point}$ と定義する。要素 $\mathsf{pair}(\mathsf{id}(A.\mathsf{Carrier}), \mathsf{refl}(A.\mathsf{point})) : E(A)$ を得る。レトラクトの列

$$\begin{array}{ll} & \sum_{Z:\mathcal{U}_\bullet(i)} E(Z) \\ \lhd & \quad \{\text{並び換え}\} \\ & \sum_{X:\mathcal{U}(i)} \sum_{e:A.\mathsf{Carrier}\simeq X} \sum_{x:X} e(A.\mathsf{point}) = x \\ \lhd & \quad \{\text{一価性}\} \\ & \sum_{x:A.\mathsf{Carrier}} A.\mathsf{point} = x \end{array}$$

を得て、最後の型は命題 3.1.4 より可縮なので、$\sum_{Z:\mathcal{U}_\bullet(i)} E(Z)$ は可縮である。

例 3.5.2 ([0028]). i を階数とする。型 $\mathsf{Magma}(i) : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- $\mathsf{Carrier} : \mathcal{U}(i)$
- $\mathsf{op} : \mathsf{Carrier} \to \mathsf{Carrier} \to \mathsf{Carrier}$

$\mathsf{Magma}(i)$ の要素は**マグマ** (magma) と呼ばれることがある。$A : \mathsf{Magma}(i)$ に対し、$E : \mathsf{Magma}(i) \to \mathcal{U}(i)$ を

$$\lambda Z.\sum_{e:A.\mathsf{Carrier}\simeq Z.\mathsf{Carrier}} \prod_{x_1,x_2:A.\mathsf{Carrier}} e((A.\mathsf{op})(x_1,x_2)) = (Z.\mathsf{op})(e(x_1), e(x_2))$$

と定義する。要素 $\mathsf{pair}(\mathsf{id}(A.\mathsf{Carrier}), \lambda x_1 x_2.\mathsf{refl}((A.\mathsf{op})(x_1,x_2))) : E(A)$ を得る。レトラクトの列

$$\begin{array}{ll} & \sum_{Z:\mathsf{Magma}(i)} E(Z) \\ \lhd & \quad \{\text{並び換え}\} \\ & \sum_{X:\mathcal{U}(i)} \sum_{e:A.\mathsf{Carrier}\simeq X} \sum_{f:X\to X\to X} \prod_{x_1,x_2} e((A.\mathsf{op})(x_1,x_2)) = f(e(x_1), e(x_2)) \\ \lhd & \quad \{\text{一価性}\} \\ & \sum_{f:A.\mathsf{Carrier}\to A.\mathsf{Carrier}\to A.\mathsf{Carrier}} \prod_{x_1,x_2} (A.\mathsf{op})(x_1,x_2) = f(x_1,x_2) \end{array}$$

を得て、最後の型は関数外延性により可縮なので、$\sum_{Z:\mathsf{Magma}(i)} E(Z)$ は可縮である。

例 3.5.3 ([002A]). i を階数とする。型 $\mathsf{ReflGraph}(i) : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- $\mathsf{Vertex} : \mathcal{U}(i)$
- $\mathsf{Edge} : \mathsf{Vertex} \to \mathsf{Vertex} \to \mathcal{U}(i)$
- $\mathsf{refl} : \prod_{x:\mathsf{Vertex}} \mathsf{Edge}(x,x)$

$\mathsf{ReflGraph}(i)$ の要素は**反射的グラフ** (reflexive graph) と呼ばれる。$A, Z : \mathsf{ReflGraph}(i)$ に

対し、$E(Z) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $v : A.\mathsf{Vertex} \simeq Z.\mathsf{Vertex}$
- $e : \prod_{[x_1,x_2:A.\mathsf{Vertex}]}(A.\mathsf{Edge})(x_1,x_2) \simeq (Z.\mathsf{Edge})(v(x_1),v(x_2))$
- $r : \prod_{x:A.\mathsf{Vertex}} e((A.\mathsf{refl})(x)) = (Z.\mathsf{refl})(v(x))$

要素 $(\!(v \equiv \mathsf{id}(_), e \equiv \lambda x_1 x_2.\mathsf{id}(_), r \equiv \lambda x.\mathsf{refl}(_))\!) : E(A)$ を得る。レトラクトの列

$$
\begin{array}{ll}
 & \sum_{Z:\mathsf{ReflGraph}(i)} E(Z) \\
\lhd & \quad \{\text{並び換え}\} \\
 & \sum_{X:\mathcal{U}(i)} \sum_{v:A.\mathsf{Vertex}\simeq X} \sum_{Y:_} \sum_{e:_} \sum_{z:_} \prod_{x:_} _ \\
\lhd & \quad \{\text{一価性}\} \\
 & \sum_{Y:A.\mathsf{Vertex}\to A.\mathsf{Vertex}\to\mathcal{U}(i)} \sum_{e:\prod_{x_1,x_2}(A.\mathsf{Edge})(x_1,x_2)\simeq Y(x_1,x_2)} \sum_{z:_} \prod_{x:_} _ \\
\lhd & \quad \{\text{関数外延性と一価性}\} \\
 & \sum_{z:\prod_{x:A.\mathsf{Vertex}}(A.\mathsf{Edge})(x,x)} \prod_{x:A.\mathsf{Vertex}} (A.\mathsf{refl})(x) = z(x)
\end{array}
$$

を得て、最後の型は関数外延性により可縮なので、$\sum_{Z:\mathsf{ReflGraph}(i)} E(Z)$ は可縮である。

数学でよく登場する構造といえば群や環などであるが、これらについての構造同一原理を正しく述べるには 5 章で導入する n **型**の概念を用いる。5.5 節でさらなる構造同一原理を見る。

3.6 同値 [000U]

同値の定義 (定義 3.1.8) が妥当なものであることを見る。同値の概念にふさわしい性質として次を示す。

- 恒等関数は同値である (補題 3.3.1)
- 同値の概念は**ホモトピー不変**である (命題 3.6.6)
- **六分の二性** (2-out-of-6 property): 合成可能な関数 f, g, h に対して、$g \circ f$ と $h \circ g$ が同値ならば $f, g, h, h \circ g \circ f$ も同値である (命題 3.6.7)

さらに、関数全体のうちの同値のなすクラスはこれらの性質を満たすものの中で最小であることを示す。つまり、任意の同値はこれらの性質のみを使って得られる (命題 3.6.8)。また、もう一つ重要な事実として、(関数外延性の下で) $\mathsf{IsEquiv}(f)$ は**命題**であるということがある (系 5.1.8)。命題は後に導入する概念なので今は説明しないが、関数が同値であることをどのように証明したかは気にしなくてよいということが分かる。

補題 3.6.1 ([002L]). i を階数、$A, B, C : \mathcal{U}(i)$ を型、$f : A \to B$ と $g : B \to C$ を型、

$c : C$ を要素とすると、$(\sum_{y:\mathsf{Fiber}(g,c)} \mathsf{Fiber}(f, y.\mathsf{elem})) \lhd\rhd \mathsf{Fiber}(g \circ f, c)$ の要素を構成できる。

証明. 次のようにわかる。

$$
\begin{array}{ll}
 & \sum_{y:\mathsf{Fiber}(g,c)} \mathsf{Fiber}(f, y.\mathsf{elem}) \\
\lhd\rhd & \quad \{\text{並び換え}\} \\
 & \sum_{x:A} \sum_{y:B} \sum_{p:f(x)=y} g(y) = c \\
\lhd\rhd & \quad \{\text{補題 3.2.3 と補題 3.2.7 と命題 3.1.4}\} \\
 & \sum_{x:A} g(f(x)) = c
\end{array}
$$

□

補題 3.6.2 ([002E]). i を階数、$A, B, C : \mathcal{U}(i)$ を型、$f : A \to B$ と $g : B \to C$ を関数とする。$f, g, g \circ f$ のうちいずれか二つが同値ならば残りの一つも同値である。つまり、次の型の要素を構成できる。

1. $\mathsf{IsEquiv}(f) \to \mathsf{IsEquiv}(g) \to \mathsf{IsEquiv}(g \circ f)$
2. $\mathsf{IsEquiv}(f) \to \mathsf{IsEquiv}(g \circ f) \to \mathsf{IsEquiv}(g)$
3. $\mathsf{IsEquiv}(g) \to \mathsf{IsEquiv}(g \circ f) \to \mathsf{IsEquiv}(f)$

証明. f が同値であると仮定すると、補題 3.6.1 と命題 3.1.6 から $\mathsf{IsEquiv}(g) \leftrightarrow \mathsf{IsEquiv}(g \circ f)$ が従う。

g と $g \circ f$ が同値であると仮定する。$b : B$ を仮定する。$r : \mathsf{Fiber}(g, g(b))$ を $(\!|\mathsf{elem} \equiv b, \mathsf{id} \equiv \mathsf{refl}(g(b))|\!)$ と定義する。レトラクト

$$
\begin{array}{ll}
 & \mathsf{Fiber}(g \circ f, g(b)) \\
\lhd\rhd & \quad \{\text{補題 3.6.1}\} \\
 & \sum_{y:\mathsf{Fiber}(g,g(b))} \mathsf{Fiber}(f, y.\mathsf{elem}) \\
\lhd\rhd & \quad \{g \text{ が同値}\} \\
 & \mathsf{Fiber}(f, r.\mathsf{elem})
\end{array}
$$

を得て、$r.\mathsf{elem} \equiv b$ であることに注意すると、$g \circ f$ が同値なので $\mathsf{Fiber}(f, b)$ は可縮である。 □

補題 3.6.3 ([002J]). i を階数、$A : \mathcal{U}(i)$ を型とする。関数 $\lambda z.\mathsf{pr}_1(z) : (\sum_{x_1:A} \sum_{x_2:A} x_1 = x_2) \to A$ は同値である。

証明. 任意の要素 $a : A$ に対して、レトラクトの列

$$
\begin{array}{ll}
 & \mathsf{Fiber}(\lambda z.\mathsf{pr}_1(z), a) \\
\lhd & \quad \{\text{並び換え}\} \\
 & \sum_{x_1:A} \sum_{p:x_1=a} \sum_{x_2:A} x_1 = x_2 \\
\lhd & \quad \{\text{補題 3.3.1}\} \\
 & \sum_{x_2:A} a = x_2
\end{array}
$$

を得て、最後の型は命題 3.1.4 により可縮である。 □

補題 3.6.4 ([002K]). i を階数、$A : \mathcal{U}(i)$ を型とする。関数 $\lambda x.\mathsf{pair}(x, \mathsf{pair}(x, \mathsf{refl}(x))) : A \to (\sum_{x_1:A} \sum_{x_2:A} x_1 = x_2)$ と $\lambda z.\mathsf{pr}_1(\mathsf{pr}_2(z)) : (\sum_{x_1:A} \sum_{x_2:A} x_1 = x_2) \to A$ は同値である。

証明. $f \equiv \lambda z.\mathsf{pr}_1(z)$, $g \equiv \lambda x.\mathsf{pair}(x, \mathsf{pair}(x, \mathsf{refl}(x)))$, $h \equiv \lambda z.\mathsf{pr}_1(\mathsf{pr}_2(z))$ と定義すると、$f \circ g \equiv \mathsf{id}(A)$ かつ $h \circ g \equiv \mathsf{id}(A)$ である。補題 3.6.3 より f は同値であるから、補題 3.6.2 と補題 3.3.1 から g も同値であると分かる。すると、再び補題 3.6.2 と補題 3.3.1 から h も同値であると分かる。 □

定義 3.6.5 ([002I]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$f, g : \prod_{x:A} B(x)$ を関数とする。型 $f \sim g : \mathcal{U}(i)$ を $\prod_{x:A} f(x) = g(x)$ と定義する。$f \sim g$ の要素を f と g の間の**ホモトピー** (homotopy) と呼ぶ。

命題 3.6.6 ([002G]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f, g : A \to B$ を関数、$p : f \sim g$ をホモトピーとする。f が同値ならば g も同値である。

証明. $q : A \to (\sum_{y_1:B} \sum_{y_2:B} y_1 = y_2)$ を $\lambda x.\mathsf{pair}(f(x), \mathsf{pair}(g(x), p(x)))$ と定義すると、$(\lambda z.\mathsf{pr}_1(z)) \circ q \equiv f$ かつ $(\lambda z.\mathsf{pr}_1(\mathsf{pr}_2(z))) \circ q \equiv g$ である。補題 3.6.2 と補題 3.6.3 と仮定から q が同値であると分かり、すると補題 3.6.2 と補題 3.6.4 から g が同値であると分かる。 □

命題 3.6.7 ([002F]). i を階数、$A, B, C, D : \mathcal{U}(i)$ を型、$f : A \to B$ と $g : B \to C$ と $h : C \to D$ を関数とする。$g \circ f$ と $h \circ g$ が同値ならば $f, g, h, h \circ g \circ f$ も同値である。

証明. 任意の要素 $d : D$ に対して、レトラクト

$$
\begin{array}{ll}
 & \mathsf{Fiber}(h \circ g \circ f, d) \\
\lhd & \quad \{\text{補題 } 3.6.1\} \\
 & \sum_{z:\mathsf{Fiber}(h,d)} \sum_{y:\mathsf{Fiber}(g,z.\mathsf{elem})} \mathsf{Fiber}(f, y.\mathsf{elem}) \\
\lhd & \quad \{\text{演習 } 2.3.13\} \\
 & \sum_{z:\mathsf{Fiber}(h,d)} \sum_{y':\mathsf{Fiber}(g,z.\mathsf{elem})} \sum_{y:\mathsf{Fiber}(g,z.\mathsf{elem})} \mathsf{Fiber}(f, y.\mathsf{elem}) \\
\lhd & \quad \{h \circ g \text{ が同値}\} \\
 & \sum_{y:\mathsf{Fiber}(g,_)} \mathsf{Fiber}(f, y.\mathsf{elem})
\end{array}
$$

を得て、最後の型は $g \circ f$ が同値なので可縮である。よって、$h \circ g \circ f$ は同値である。すると補題 3.6.2 より残りの関数もすべて同値である。 □

命題 3.6.8 ([002H]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。f が同値ならば、関数 $g, h : B \to A$ とホモトピー $p : f \circ g \sim \mathsf{id}(B)$ と $q : h \circ f \sim \mathsf{id}(A)$ を構成できる。

証明. f が同値であると仮定する。関数 $G : \prod_{y:B} \mathsf{Fiber}(f, y)$ と同一視 $P : \prod_{y:B} \prod_{z:\mathsf{Fiber}(f,y)} G(y) = z$ を得る。$g \equiv \lambda y.G(y).\mathsf{elem}$, $p \equiv \lambda y.G(y).\mathsf{id}$ と定義する。$r : \prod_{x:A} \mathsf{Fiber}(f, f(x))$ を $r \equiv \lambda x.(\!|\mathsf{elem} \equiv x, \mathsf{id} \equiv \mathsf{refl}(f(x))|\!)$ と定義する。$h \equiv g$ と定義し、$q \equiv \lambda x.\mathsf{ap}(\lambda z.z.\mathsf{id})(P(f(x), r(x)))$ と定義すればよい。 □

3.6.1 他の同値の概念 [004I]

定義 3.1.8 の他にも同値の概念の定義はある。

定義 3.6.9 ([004J]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。

- 型 $\mathsf{LInv}(f) : \mathcal{U}(i)$ を $\{\!|\mathsf{inv} : B \to A, \mathsf{is\text{-}linv} : \mathsf{inv} \circ f \sim \mathsf{id}(A)|\!\}$ と定義する。
- 型 $\mathsf{RInv}(f) : \mathcal{U}(i)$ を $\{\!|\mathsf{inv} : B \to A, \mathsf{is\text{-}rinv} : f \circ \mathsf{inv} \sim \mathsf{id}(B)|\!\}$ と定義する。
- 型 $\mathsf{IsBiinv}(f) : \mathcal{U}(i)$ を $\{\!|\mathsf{linv} : \mathsf{LInv}(f), \mathsf{rinv} : \mathsf{RInv}(f)|\!\}$ と定義する。

$\mathsf{IsBiinv}(f)$ の要素がある時、f は**両側可逆** (biinvertible) であると言う。

命題 3.6.10 ([004K]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。型 $\mathsf{IsEquiv}(f)$ と $\mathsf{IsBiinv}(f)$ は論理的に同値である。

証明. 補題 3.3.1 と命題 3.6.6 と命題 3.6.7 と命題 3.6.8 から従う。 □

定義 3.6.9 の利点は比較的低級な概念で定義できる点である。この定義で使われている概念は恒等関数、合成、ホモトピーだけである。これらの概念は一般の高次圏で意味をなすものであり、実際、高次圏での同値は両側可逆性で定義できる。一方、ファイバーや

可縮性には型理論の言語が本質的に使われている。ただ、定義 3.6.9 から直接取り出せる情報はそれほど有益でなく、逆関数の取り方も二種類あるので使い勝手が良いとは言えない。

定義 3.6.11 ([004L]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。型 $\mathsf{IsHAE}(f) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{inv} : B \to A$
- $\mathsf{unit} : \mathsf{inv} \circ f \sim \mathsf{id}(A)$
- $\mathsf{counit} : f \circ \mathsf{inv} \sim \mathsf{id}(B)$
- $\mathsf{coh} : \prod_{x:A} f(\mathsf{unit}(x)) = \mathsf{counit}(f(x))$

$\mathsf{IsHAE}(f)$ の要素がある時、f は**半随伴同値** (half adjoint equivalence) であると言う。

演習 3.6.12 ([004M]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。型 $\mathsf{IsEquiv}(f)$ と $\mathsf{IsHAE}(f)$ は論理的に同値であることを示せ。(ヒント: 命題 3.6.8 の証明を拡張すれば $\mathsf{IsEquiv}(f) \to \mathsf{IsHAE}(f)$ を示せる。)

定義 3.6.11 は定義 3.6.9 と比べて有益な情報が多い。その分、その定義には一つ高次の同一視 (coh) を使う。

次の定義 3.6.13 も同値の概念を定義しているように見えるかもしれないが、これは正しい同値の定義**ではない**。

定義 3.6.13 ([004T]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。型 $\mathsf{QInv}(f) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{inv} : B \to A$
- $\mathsf{unit} : \mathsf{inv} \circ f \sim \mathsf{id}(A)$
- $\mathsf{counit} : f \circ \mathsf{inv} \sim \mathsf{id}(B)$

命題 3.6.14 ([004U]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。型 $\mathsf{IsEquiv}(f)$ と $\mathsf{QInv}(f)$ は論理的に同値である。

証明. $\mathsf{IsHAE}(f) \to \mathsf{QInv}(f)$ と $\mathsf{QInv}(f) \to \mathsf{IsBiinv}(f)$ は容易に示せるので、命題 3.6.10 と演習 3.6.12 を使う。 □

命題 3.6.14 により、関数 f が同値であることを示す目的では $\mathsf{QInv}(f)$ を使って何も問題はない。$\mathsf{QInv}(f)$ が他に挙げた正しい同値の概念と決定的に違うのは、同一でない二つの $\mathsf{QInv}(f)$ の要素がありうる点である。後に示すが、(関数外延性の下で)$\mathsf{IsEquiv}(f)$ と $\mathsf{IsBiinv}(f)$ と $\mathsf{IsHAE}(f)$ はいずれも任意の二つの要素が同一視される (系 5.1.8 と命題 5.1.12 と命題 5.1.14)。$\mathsf{QInv}(f)$ についてはこの性質は示せないどころか、一価性公

理の下で $\mathsf{QInv}(f)$ が同一でない二つの要素を持つような f を構成できることが知られている。

第 4 章

高次帰納的型 [0003]

4.1 始代数 [0031]

定義 4.1.1 ([0035]). i を階数とする。型 $\mathbb{N}\text{-}\mathsf{Alg}(i) : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- $\mathsf{Carrier} : \mathcal{U}(i)$
- $\mathsf{zero} : \mathsf{Carrier}$
- $\mathsf{suc} : \mathsf{Carrier} \to \mathsf{Carrier}$

$\mathbb{N}\text{-}\mathsf{Alg}(i)$ の要素を (階数 i の) $\mathbb{N}$ **代数** ($\mathbb{N}$-algebra) と呼ぶ。

定義 4.1.2 ([0036]). i を階数、$A, B : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数とする。型 $\mathsf{Hom}(A, B) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{carrier} : A.\mathsf{Carrier} \to B.\mathsf{Carrier}$
- $\mathsf{zero} : \mathsf{carrier}(A.\mathsf{zero}) = B.\mathsf{zero}$
- $\mathsf{suc} : \prod_{x : A.\mathsf{Carrier}} \mathsf{carrier}((A.\mathsf{suc})(x)) = (B.\mathsf{suc})(\mathsf{carrier}(x))$

$\mathsf{Hom}(A, B)$ の要素を A から B への $\mathbb{N}$ **準同型写像** ($\mathbb{N}$-homomorphism) と呼ぶ。

演習 4.1.3 ([0037]). i を階数とする。

1. $\mathbb{N}$ 代数 $A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ に対して、**恒等射** $\mathsf{id}(A) : \mathsf{Hom}(A, A)$ を構成せよ。
2. $\mathbb{N}$ 代数 $A, B, C : \mathbb{N}\text{-}\mathsf{Alg}(i)$ と $\mathbb{N}$ 準同型写像 $f : \mathsf{Hom}(A, B)$ と $g : \mathsf{Hom}(B, C)$ に対して、**合成** $g \circ f : \mathsf{Hom}(A, C)$ を構成せよ。

定義 4.1.4 ([0038]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数とする。型 $\mathsf{IsInit}(A) : \mathcal{U}(\mathsf{suc}(i))$ を $\prod_{X : \mathbb{N}\text{-}\mathsf{Alg}(i)} \mathsf{IsContr}(\mathsf{Hom}(A, X))$ と定義する。$\mathsf{IsInit}(A)$ の要素がある時、A は**始** $\mathbb{N}$ **代数** (initial $\mathbb{N}$-algebra) であると言う。

定義 4.1.5 ([0039]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数とする。型 $\mathbb{N}\text{-}\mathsf{Alg}_{/A} : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- $\mathsf{Carrier} : A.\mathsf{Carrier} \to \mathcal{U}(i)$
- $\mathsf{zero} : \mathsf{Carrier}(A.\mathsf{zero})$
- $\mathsf{suc} : \prod_{[x:A.\mathsf{Carrier}]} \mathsf{Carrier}(x) \to \mathsf{Carrier}((A.\mathsf{suc})(x))$

$\mathbb{N}\text{-}\mathsf{Alg}_{/A}$ の要素を A **上の** $\mathbb{N}$ **代数** ($\mathbb{N}$-algebra over A) と呼ぶ。

定義 4.1.6 ([003A]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数、$B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ を A 上の $\mathbb{N}$ 代数とする。型 $\mathsf{Sect}(B) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{carrier} : \prod_{x:A.\mathsf{Carrier}} (B.\mathsf{Carrier})(x)$
- $\mathsf{zero} : \mathsf{carrier}(A.\mathsf{zero}) = B.\mathsf{zero}$
- $\mathsf{suc} : \prod_{x:A.\mathsf{Carrier}} \mathsf{carrier}((A.\mathsf{suc})(x)) = (B.\mathsf{suc})(\mathsf{carrier}(x))$

$\mathsf{Sect}(B)$ の要素を B の**切断** (section) と呼ぶ。

例 4.1.7 ([003B]). i を階数、$A, B : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数とする。A 上の $\mathbb{N}$ 代数 $A^*B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ を次のように定義する。

- $\mathsf{Carrier} \equiv \lambda x.B.\mathsf{Carrier}$
- $\mathsf{zero} \equiv B.\mathsf{zero}$
- $\mathsf{suc} \equiv \lambda xy.(B.\mathsf{suc})(y)$

定義から $\mathsf{Sect}(A^*B) \equiv \mathsf{Hom}(A, B)$ である。

例 4.1.8 ([003G]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数、$B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ を A 上の $\mathbb{N}$ 代数、$f, g : \mathsf{Sect}(B)$ を切断とする。A 上の $\mathbb{N}$ 代数 $\mathsf{Id}(f, g) : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ を次のように定義する。

- $\mathsf{Carrier} \equiv \lambda x.(f.\mathsf{carrier})(x) = (g.\mathsf{carrier})(x)$
- $\mathsf{zero} \equiv (g.\mathsf{zero})^{-1} \circ f.\mathsf{zero}$
- $\mathsf{suc} \equiv \lambda xy.(g.\mathsf{suc})(x)^{-1} \circ (B.\mathsf{suc})(y) \circ (f.\mathsf{suc})(x)$

演習 4.1.9 ([003H]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数、$B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ を A 上の $\mathbb{N}$ 代数、$f : \mathsf{Sect}(B)$ を切断とする。

1. $\mathsf{Sect}(\mathsf{Id}(f, f))$ の要素を構成せよ。
2. 関数外延性を仮定すると $\sum_{g:\mathsf{Sect}(B)} \mathsf{Sect}(\mathsf{Id}(f, g))$ は可縮であることを示せ。

よって、定理 3.2.4 より、関数外延性の下で同値 $(f = g) \simeq \mathsf{Sect}(\mathsf{Id}(f, g))$ を得る。

定義 4.1.10 ([003D]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数、$B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ を A 上の $\mathbb{N}$ 代

数とする。$\mathbb{N}$ 代数 $\mathsf{Total}(B) : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を次のように定義する。

- $\mathsf{Carrier} \equiv \sum_{x:A.\mathsf{Carrier}}(B.\mathsf{Carrier})(x)$
- $\mathsf{zero} \equiv \mathsf{pair}(A.\mathsf{zero}, B.\mathsf{zero})$
- $\mathsf{suc} \equiv \lambda z.\mathsf{pair}((A.\mathsf{suc})(\mathsf{pr}_1(z)), (B.\mathsf{suc})(\mathsf{pr}_2(z)))$

また、$\mathbb{N}$ 準同型写像 $\mathsf{pr}(B) : \mathsf{Hom}(\mathsf{Total}(B), A)$ を次のように定義する。

- $\mathsf{carrier} \equiv \lambda z.\mathsf{pr}_1(z)$
- $\mathsf{zero} \equiv \mathsf{refl}(_)$
- $\mathsf{suc} \equiv \lambda z.\mathsf{refl}(_)$

演習 4.1.11 ([003E]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数、$B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ を A 上の $\mathbb{N}$ 代数とする。$\mathsf{Sect}(B)$ は $\mathsf{Fiber}(\lambda(f : \mathsf{Hom}(A, \mathsf{Total}(B))).\mathsf{pr}(B) \circ f, \mathsf{id}(A))$ のレトラクトであることを示せ。

演習 4.1.12 ([003F]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。A と B が可縮ならば f は同値であることを示せ。

定理 4.1.13 ([003C]). i を階数、$A : \mathbb{N}\text{-}\mathsf{Alg}(i)$ を $\mathbb{N}$ 代数とする。次の型は論理的に同値である。

1. $\mathsf{IsInit}(A)$
2. $\prod_{X:\mathbb{N}\text{-}\mathsf{Alg}_{/A}} \mathsf{IsContr}(\mathsf{Sect}(X))$

さらに、関数外延性を仮定すると、これらは $\prod_{X:\mathbb{N}\text{-}\mathsf{Alg}_{/A}} \mathsf{Sect}(X)$ と論理的に同値である。

証明. 2 から 1 は例 4.1.7 から従う。

1 から 2 を示す。$B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ とする。仮定より、$\mathsf{Hom}(A, \mathsf{Total}(B))$ と $\mathsf{Hom}(A, A)$ は可縮である。よって、演習 4.1.12 より、関数 $\lambda f.\mathsf{pr}(B) \circ f : \mathsf{Hom}(A, \mathsf{Total}(B)) \to \mathsf{Hom}(A, A)$ は同値である。すると、演習 4.1.11 と命題 3.1.6 より $\mathsf{Sect}(B)$ は可縮である。

関数外延性を仮定し、$h : \prod_{X:\mathbb{N}\text{-}\mathsf{Alg}_{/A}} \mathsf{Sect}(X)$ を仮定する。$B : \mathbb{N}\text{-}\mathsf{Alg}_{/A}$ に対して、$\mathsf{Sect}(B)$ は可縮であることを示す。$h(B) : \mathsf{Sect}(B)$ を得る。任意の $f : \mathsf{Sect}(B)$ に対して、同一視 $p(f) : h(B) = f$ を構成すればよい。演習 4.1.9 より、$p'(f) : \mathsf{Sect}(\mathsf{Id}(h(B), f))$ を構成すればよいが、$p'(f) \equiv h(\mathsf{Id}(h(B), f))$ とすればよい。 □

系 4.1.14 ([003I]). 関数外延性を仮定すると、$\mathbb{N} : \mathcal{U}(0)$ と $0 : \mathbb{N}$ と $\lambda x.\mathsf{suc}(x) : \mathbb{N} \to \mathbb{N}$ は始 $\mathbb{N}$ 代数を定める。 □

4.2　円周 [0032]

定義 4.2.1 ([003J]). i を階数とする。型 $\mathbb{S}^1\text{-}\mathsf{Alg}(i) : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- $\mathsf{Carrier} : \mathcal{U}(i)$
- $\mathsf{base} : \mathsf{Carrier}$
- $\mathsf{loop} : \mathsf{base} = \mathsf{base}$

$\mathbb{S}^1\text{-}\mathsf{Alg}(i)$ の要素を (階数 i の) $\mathbb{S}^1$ **代数** ($\mathbb{S}^1$-algebra) と呼ぶ。

定義 4.2.2 ([003L]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$a_1, a_2 : A$ を要素、$p : a_1 = a_2$ を同一視、$b_1 : B(a_1)$ と $b_2 : B(a_2)$ を要素とする。型 $b_1 =_p^B b_2 : \mathcal{U}(i)$ を $\mathsf{transport}(B, p)(b_1) = b_2$ と定義する。$b_1 =_p^B b_2$ の要素を b_1 と b_2 の p **上の同一視** (identification over p) と呼ぶ。

定義 4.2.3 ([003K]). i を階数、$A, B : \mathbb{S}^1\text{-}\mathsf{Alg}(i)$ を $\mathbb{S}^1$ 代数とする。型 $\mathsf{Hom}(A, B) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{carrier} : A.\mathsf{Carrier} \to B.\mathsf{Carrier}$
- $\mathsf{base} : \mathsf{carrier}(A.\mathsf{base}) = B.\mathsf{base}$
- $\mathsf{loop} : \mathsf{carrier}(A.\mathsf{loop}) =_{\mathsf{base}}^{\lambda x.x=x} B.\mathsf{loop}$

$\mathsf{Hom}(A, B)$ の要素を $\mathbb{S}^1$ **準同型写像** ($\mathbb{S}^1$-homomorphism) と呼ぶ。

定義 4.2.4 ([003N]). i を階数、$A : \mathbb{S}^1\text{-}\mathsf{Alg}(i)$ を $\mathbb{S}^1$ 代数とする。型 $\mathsf{IsInit}(A) : \mathcal{U}(\mathsf{suc}(i))$ を $\prod_{X : \mathbb{S}^1\text{-}\mathsf{Alg}(i)} \mathsf{IsContr}(\mathsf{Hom}(A, X))$ と定義する。$\mathsf{IsInit}(A)$ の要素がある時、A は**始** $\mathbb{S}^1$ **代数** (initial $\mathbb{S}^1$-algebra) であると言う。

公理 4.2.5 ([003M]).
1. 始 $\mathbb{S}^1$ 代数 $\mathbb{S}^1 : \mathbb{S}^1\text{-}\mathsf{Alg}(0)$ がある。
2. i を階数、$A : \mathbb{S}^1\text{-}\mathsf{Alg}(i)$ を始 $\mathbb{S}^1$ 代数とすると、A は階数 $\mathsf{suc}(i)$ においても始 $\mathbb{S}^1$ 代数である。

4.3　押し出し [0033]

定義 4.3.1 ([003O]). i を階数、$A, B, C : \mathcal{U}(i)$ を型 $f : C \to A$ と $g : C \to B$ を関数とする。型 $\mathsf{PO\text{-}Alg}(f, g) : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- $\mathsf{Carrier} : \mathcal{U}(i)$

- $\mathsf{in}_1 : A \to \mathsf{Carrier}$
- $\mathsf{in}_2 : B \to \mathsf{Carrier}$
- $\mathsf{glue} : \prod_{z:C} \mathsf{in}_1(f(z)) = \mathsf{in}_2(g(z))$

$\mathsf{PO\text{-}Alg}(f,g)$ の要素を $PO(f,g)$ **代数** ($\mathsf{PO}(f,g)$-algebra) と呼ぶ。

定義 4.3.2 ([003S]). i を階数、$A,B : \mathcal{U}(i)$ を型、$C : A \to B \to \mathcal{U}(i)$ を型の族、$a_1,a_2 : A$ と $b_1,b_2 : B$ を要素、$p : a_1 = a_2$ と $q : b_1 = b_2$ を同一視、$c_1 : C(a_1,b_1)$ と $c_2 : C(a_2,b_2)$ を要素とする。型 $c_1 =^{C}_{p,q} c_2 : \mathcal{U}(i)$ を $\mathsf{transport}(\lambda y.C(a_1,y),q)(c_1) =^{\lambda x.C(x,b_2)}_{p} c_2$ と定義する。

定義 4.3.3 ([003P]). i を階数、$A,B,C : \mathcal{U}(i)$ を型、$f : C \to A$ と $g : C \to B$ を関数、$D,E : \mathsf{PO\text{-}Alg}(f,g)$ を $\mathsf{PO}(f,g)$ 代数とする。型 $\mathsf{Hom}(D,E) : \mathcal{U}(i)$ を次のレコード型と定義する。

- $\mathsf{carrier} : D.\mathsf{Carrier} \to E.\mathsf{Carrier}$
- $\mathsf{in}_1 : \prod_{x:A} \mathsf{carrier}((D.\mathsf{in}_1)(x)) = (E.\mathsf{in}_1)(x)$
- $\mathsf{in}_2 : \prod_{y:B} \mathsf{carrier}((D.\mathsf{in}_2)(y)) = (E.\mathsf{in}_2)(y)$
- $\mathsf{glue} : \prod_{z:C} \mathsf{carrier}((D.\mathsf{glue})(z)) =^{\lambda xy.x=y}_{\mathsf{in}_1(f(z)),\mathsf{in}_2(g(z))} (E.\mathsf{glue})(z)$

$\mathsf{Hom}(D,E)$ の要素を D から E への $PO(f,g)$ **準同型写像** ($\mathsf{PO}(f,g)$-homomorphism) と呼ぶ。

定義 4.3.4 ([003Q]). i を階数、$A,B,C : \mathcal{U}(i)$ を型、$f : C \to A$ と $g : C \to B$ を関数、$D : \mathsf{PO\text{-}Alg}(f,g)$ を $\mathsf{PO}(f,g)$ 代数とする。型 $\mathsf{IsInit}(D) : \mathcal{U}(\mathsf{suc}(i))$ を $\prod_{X:\mathsf{PO\text{-}Alg}(f,g)} \mathsf{IsContr}(\mathsf{Hom}(D,X))$ と定義する。$\mathsf{IsInit}(D)$ の要素がある時、D は**始** $PO(f,g)$ **代数** (initial $\mathsf{PO}(f,g)$-algebra) であると言う。

公理 4.3.5 ([003R]). i を階数、$A,B,C : \mathcal{U}(i)$ を型 $f : C \to A$ と $g : C \to B$ を関数とする。

- 始 $\mathsf{PO}(f,g)$ 代数 $\mathsf{PO}(f,g) : \mathsf{PO\text{-}Alg}(f,g)$ がある。
- j を階数、$D : \mathsf{PO\text{-}Alg}(f,g)$ を階数 j における始 $\mathsf{PO}(f,g)$ 代数とすると、D は階数 $\mathsf{suc}(j)$ においても始 $\mathsf{PO}(f,g)$ 代数である。

4.4 降下性 [0034]

高次帰納的型は一価性公理の下で**降下性** (descent property) という著しい性質を持つ。降下性という言葉は∞トポス理論から取ったもので、余極限が有限極限に関して「良く」

振る舞うという性質である。

最も簡単な降下性は空型 $\mathbf{0}$ が**厳格** (strict) であることを表す次の命題 4.4.1 であり、これには一価性は必要ない。

命題 4.4.1 ([003T]). i を階数、$A : \mathcal{U}(i)$ を型、$f : A \to \mathbf{0}$ を関数とする。f は同値である。

証明. $\mathbf{0}$ の帰納法による。 □

次の命題 4.4.2 は余積の**広域性** (extensivity) と呼ばれる性質で、これにも一価性は必要ない。

命題 4.4.2 ([003U]). i を階数、$A, B : \mathcal{U}(i)$ を型、$P : A \to \mathcal{U}(i)$ と $Q : B \to \mathcal{U}(i)$ を型の族とする。型の族 $R : A + B \to \mathcal{U}(i)$ を余積の帰納法により $R(\mathsf{in}_1(x)) \equiv P(x)$ と $R(\mathsf{in}_2(y)) \equiv Q(y)$ と定義する。この時、関数 $\lambda u.\mathsf{pair}(\mathsf{in}_1(\mathsf{pr}_1(u)), \mathsf{pr}_2(u)) : (\sum_{x:A} P(x)) \to (\sum_{z:A+B} R(z))$ と $\lambda v.\mathsf{pair}(\mathsf{in}_2(\mathsf{pr}_1(v)), \mathsf{pr}_2(v)) : (\sum_{y:B} Q(y)) \to (\sum_{z:A+B} R(z))$ から余積の帰納法で定まる関数

$$(\textstyle\sum_{x:A} P(x)) + (\sum_{y:B} Q(y)) \to (\sum_{z:A+B} R(z))$$

は同値である。

証明. $\sum_{z:A+B} R(z)$ が余積の帰納法原理を満たすことを示す。次を仮定する。

- 型の族 $T : (\sum_{z:A+B} R(z)) \to \mathcal{U}(i)$
- 関数 $f : \prod_{u:\sum_{x:A} P(x)} T(\mathsf{pair}(\mathsf{in}_1(\mathsf{pr}_1(u)), \mathsf{pr}_2(u)))$
- 関数 $g : \prod_{v:\sum_{y:B} Q(y)} T(\mathsf{pair}(\mathsf{in}_2(\mathsf{pr}_1(v)), \mathsf{pr}_2(v)))$

カリー化により、関数 $h : \prod_{z:A+B} \prod_{w:R(z)} T(\mathsf{pair}(z, w))$ を構成することになるが、$A+B$ の帰納法により、$h(\mathsf{in}_1(x)) \equiv \lambda u.f(\mathsf{pair}(x, u))$ と $h(\mathsf{in}_2(y)) \equiv \lambda v.g(\mathsf{pair}(y, v))$ と定義すればよい。 □

定理 4.4.3 ([003V]). i を階数、$A, B, C : \mathcal{U}(i)$ を型、$f : C \to A$ と $g : C \to B$ を関数、$P : A \to \mathcal{U}(i)$ と $Q : B \to \mathcal{U}(i)$ を型の族、$e : \prod_{[z:C]} P(f(z)) \simeq Q(g(z))$ を同値の族、$D : \mathsf{PO\text{-}Alg}(f, g)$ を始 $\mathsf{PO}(f, g)$ 代数とする。

1. $\mathcal{U}(i)$ が一価性を満たすと仮定すると、次を構成できる。
   - 型の族 $G : D.\mathsf{Carrier} \to \mathcal{U}(i)$
   - 同値の族 $j_1 : \prod_{[x:A]} P(x) \simeq G((D.\mathsf{in}_1)(x))$
   - 同値の族 $j_2 : \prod_{[y:B]} Q(y) \simeq G((D.\mathsf{in}_2)(y))$
   - 同一視の族 $\theta : \prod_{[z:C]} \prod_{p:P(f(z))} j_1(p) =^G_{(D.\mathsf{glue})(z)} j_2(e(p))$

2. $\overline{f} : (\sum_{z:C} P(f(z))) \to (\sum_{x:A} P(x))$ を $\lambda z.\mathsf{pair}(f(\mathsf{pr}_1(z)), \mathsf{pr}_2(z))$ と、$\overline{g} : (\sum_{z:C} P(f(z))) \to (\sum_{y:B} Q(y))$ を $\lambda z.\mathsf{pair}(g(\mathsf{pr}_1(z)), e(\mathsf{pr}_2(z)))$ と定義する。1 のような構造 (G, j_1, j_2, θ) に対して、次の構造は始 $\mathsf{PO}(\overline{f}, \overline{g})$ 代数を定める。
   - $\mathsf{Carrier} \equiv \sum_{w:D} G(w)$
   - $\mathsf{in}_1 \equiv \lambda x.\mathsf{pair}((D.\mathsf{in}_1)(\mathsf{pr}_1(x)), j_1(\mathsf{pr}_2(x)))$
   - $\mathsf{in}_2 \equiv \lambda y.\mathsf{pair}((D.\mathsf{in}_2)(\mathsf{pr}_1(y)), j_2(\mathsf{pr}_2(y)))$
   - $\mathsf{glue} \equiv \lambda z.\mathsf{pair}^{=}((D.\mathsf{glue})(\mathsf{pr}_1(z)), \theta(\mathsf{pr}_2(z)))$

証明. 1 を示す。一価性より、e から同一視の族 $e' : \prod_{z:C} P(f(z)) = Q(g(z))$ を得るので、始 $\mathsf{PO}(f, g)$ 代数の帰納法により目的のものを構成できる。

2 の証明は命題 4.4.2 と発想は同じである。 □

第 5 章

n 型 [003W]

定義 5.0.1 ([003X]). 1. 型 TruncLevel : $\mathcal{U}(0)$ を $\mathbb{N} + (\mathbf{1} + \mathbf{1})$ と定義する。
2. 要素 -2 : TruncLevel を $\mathsf{in}_2(\mathsf{in}_2(\star))$ と定義する。
3. 要素 -1 : TruncLevel を $\mathsf{in}_2(\mathsf{in}_1(\star))$ と定義する。
4. 要素 n : TruncLevel に対して、要素 $\mathsf{suc}(n)$: TruncLevel を $\mathsf{suc}(-2) \equiv -1$, $\mathsf{suc}(-1) \equiv \mathsf{in}_1(0)$, $\mathsf{suc}(\mathsf{in}_1(n)) \equiv \mathsf{in}_1(\mathsf{suc}(n))$ と定義する。

要素 $n : \mathbb{N}$ に対しては、$\mathsf{in}_1(\ )$ を省略して n 自身を TruncLevel の要素とみなす。

定義 5.0.2 ([003Y]). i を階数、$A : \mathcal{U}(i)$ を型とする。n : TruncLevel に対して、型 $\mathsf{IsTrunc}(n, A) : \mathcal{U}(i)$ を次のように定義する。

- $\mathsf{IsTrunc}(-2, A) \equiv \mathsf{IsContr}(A)$
- $\mathsf{IsTrunc}(\mathsf{suc}(n), A) \equiv \prod_{x_1, x_2 : A} \mathsf{IsTrunc}(n, x_1 = x_2)$

$\mathsf{IsTrunc}(n, A)$ の要素がある時、A は n **型** (n-type) である、または n-truncated であると言う。

定義 5.0.3 ([0053]). i を階数、n : TruncLevel を要素とする。型 $\langle n \rangle$-$\mathsf{Type}(i) : \mathcal{U}(\mathsf{suc}(i))$ を $\{\!\{\mathsf{Type} : \mathcal{U}(i), \mathsf{is\text{-}trunc} : \mathsf{IsTrunc}(n, \mathsf{Type})\}\!\}$ と定義する。

定義 5.0.4 ([0046]). i を階数、$A, B : \mathcal{U}(i)$ を型、$r : \mathsf{Retract}(A, B)$ を要素、$a_1, a_2 : A$ を要素とする。要素 $\mathsf{ret\text{-}id}(r, a_1, a_2) : \mathsf{Retract}(a_1 = a_2, (r.\mathsf{section})(a_1) = (r.\mathsf{section})(a_2))$ を次のように構成する。$f \equiv r.\mathsf{section}$, $g \equiv r.\mathsf{retraction}$, $p \equiv r.\mathsf{r\text{-}s}$ と定義する。$F : a_1 = a_2 \to f(a_1) = f(a_2)$ を $\mathsf{ap}(f)$ と定義する。$G : f(a_1) = f(a_2) \to a_1 = a_2$ を $\lambda q.(p(a_2) \circ \mathsf{ap}(g)(q)) \circ p(a_1)^{-1}$ と定義する。$\prod_{z : a_1 = a_2} G(F(z)) = z$ を示すには、同一視型の帰納法により $G(F(\mathsf{refl}(a_1))) = \mathsf{refl}(a_1)$ を示せばよいが、$G(F(\mathsf{refl}(a_1))) \equiv p(a_1) \circ p(a_1)^{-1}$ なので演習 2.4.13 を適用すればよい。

命題 5.0.5 ([0045]). i を階数、$A, B : \mathcal{U}(i)$ を型、$r : \mathsf{Retract}(A, B)$ と n : TruncLevel

を要素とする。B が n 型ならば、A も n 型である。

証明. n についての帰納法による。n が -2 の時は命題 3.1.6 による。

n について主張が成り立つと仮定し、$\mathsf{suc}(n)$ の場合を示す。$x_1, x_2 : A$ に対して、$\mathsf{IsTrunc}(n, x_1 = x_2)$ を示せばよいが、定義 5.0.4 と帰納法の仮定から直ちに従う。 □

命題 5.0.6 ([0048]). 関数外延性を仮定する。i を階数、$A : \mathcal{U}(i)$ を型 $B : A \to \mathcal{U}(i)$ を型の族、$n : \mathsf{TruncLevel}$ を要素とする。$\prod_{x:A} \mathsf{IsTrunc}(n, B(x))$ の要素があるならば、$\prod_{x:A} B(x)$ は n 型である。

証明. n についての帰納法による。n が -2 の場合は命題 3.4.3 による。

n の場合に主張が成り立つと仮定し、$\mathsf{suc}(n)$ の場合を示す。$f, g : \prod_{x:A} B(x)$ に対し、$f = g$ が n 型であることを示す。関数外延性より、同値 $(f = g) \simeq (\prod_{x:A} f(x) = g(x))$ を得る。各 $f(x) = g(x)$ は B についての仮定より n 型であるから、帰納法の仮定と命題 5.0.5 から $f = g$ は n 型である。 □

命題 5.0.7 ([004X]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$n : \mathsf{TruncLevel}$ を要素とする。A が n 型で、任意の $x : A$ に対して $B(x)$ が n 型ならば、$\sum_{x:A} B(x)$ も n 型である。

証明. n についての帰納法による。n が -2 の場合は容易である。

n の場合に主張が成り立つと仮定し、$\mathsf{suc}(n)$ の場合を示す。$c_1, c_2 : \sum_{x:A} B(x)$ に対し、$c_1 = c_2$ が n 型であることを示す。系 3.2.5 より、同値

$$(c_1 = c_2) \simeq \sum_{z:\mathsf{pr}_1(c_1)=\mathsf{pr}_1(c_2)} \mathsf{transport}(B, z)(\mathsf{pr}_2(c_1)) = \mathsf{pr}_2(c_2)$$

を得る。仮定より、$\mathsf{pr}_1(c_1) = \mathsf{pr}_1(c_2)$ と各 $\mathsf{transport}(B, z)(\mathsf{pr}_2(c_1)) = \mathsf{pr}_2(c_2)$ は n 型である。よって、帰納法の仮定を適用すればよい。 □

命題 5.0.8 ([004F]). i を階数、$A : \mathcal{U}(i)$ を型、$n : \mathsf{TruncLevel}$ を要素とする。A が n 型ならば、A は $\mathsf{suc}(n)$ 型である。

証明. n についての帰納法による。n が -2 の場合は命題 3.1.3 による。

n の場合に主張が成り立つと仮定して、$\mathsf{suc}(n)$ の場合を示す。A が $\mathsf{suc}(n)$ 型と仮定して、任意の $x_1, x_2 : A$ に対して $x_1 = x_2$ が $\mathsf{suc}(n)$ 型であることを示せばよいが、仮定と帰納法の過程から直ちに従う。 □

系 5.0.9 ([0052]). i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2 : A$ を要素、$n : \mathsf{TruncLevel}$ を要素とする。A が n 型ならば、$a_1 = a_2$ も n 型である。 □

5.1 命題 [003Z]

定義 5.1.1 ([0040]). i を階数、$A : \mathcal{U}(i)$ を型とする。型 $\mathsf{IsProp}(A) : \mathcal{U}(i)$ を $\mathsf{IsTrunc}(-1, A)$ と定義する。$\mathsf{IsProp}(A)$ の要素がある時、A は**命題** (proposition) であると言う。

例 5.1.2 ([004G]). $\mathbf{0}$ は命題である。実際、帰納法で直ちに示せる。

命題 5.1.3 ([0041]). i を階数、$A : \mathcal{U}(i)$ を型とする。次の型は論理的に同値である。

1. $\mathsf{IsProp}(A)$
2. $\prod_{x_1, x_2 : A} x_1 = x_2$
3. $A \to \mathsf{IsContr}(A)$

証明. 1 から 2 は定義からすぐである。

2 から 3 を示す。$H : \prod_{x_1, x_2 : A} x_1 = x_2$ と $a : A$ を仮定する。a があるので、A が可縮であることを示すには $\prod_{x:A} a = x$ の要素を構成すればよいが、$\lambda x.H(a, x)$ でよい。

3 から 1 を示す。$H : A \to \mathsf{IsContr}(A)$ と $x_1, x_2 : A$ を仮定する。$x_1 = x_2$ が可縮であることを示すが、$H(x_1) : \mathsf{IsContr}(A)$ があるので、命題 3.1.3 を適用すればよい。 □

命題 5.1.4 ([0049]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$c_1, c_2 : \sum_{x:A} B(x)$ を要素とする。$\prod_{x:A} \mathsf{IsProp}(B(x))$ の要素があるならば、同値 $(c_1 = c_2) \simeq (\mathsf{pr}_1(c_1) = \mathsf{pr}_1(c_2))$ を得る。

証明. 定理 3.2.4 を適用する。$\sum_{z : \sum_{x:A} B(x)} \mathsf{pr}_1(c_1) = \mathsf{pr}_1(z)$ が可縮であることを示す。レトラクトの列

$$
\begin{array}{ll}
 & \sum_{z : \sum_{x:A} B(x)} \mathsf{pr}_1(c_1) = \mathsf{pr}_1(z) \\
\lhd & \quad \{\text{並び換え}\} \\
 & \sum_{x:A} \sum_{w : \mathsf{pr}_1(c_1) = x} B(x) \\
\lhd & \quad \{\text{命題 } 3.1.4\} \\
 & B(\mathsf{pr}_1(c_1))
\end{array}
$$

を得て、最後の型は要素 $\mathsf{pr}_2(c_1)$ を持つので仮定と命題 5.1.3 より可縮である。 □

命題 5.1.4 より、各 $B(x)$ が命題の時は、$\sum_{x:A} B(x)$ の要素の同一視に関しては二番目の要素は完全に無視される。そのため、$\sum_{x:A} B(x)$ は要素 $a : A$ と要素 $b : B(a)$ の対のなす型というよりは、要素 $a : A$ であって**性質** $B(a)$ を満たすもののなす A の**部分型**であると考えられる。この視点を強調するために記法を導入する。

記法 5.1.5 ([004A]). i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族とする。関数外延性の下で $\prod_{x:A} \mathsf{IsProp}(B(x))$ の要素がある時、$\sum_{x:A} B(x)$ のことを $\{x : A \mid B(x)\}$ と書くことがある。さらに、要素 $c : \{x : A \mid B(x)\}$ に対して、$\mathsf{pr}_1(\ )$ を省略して c そのものを A の要素とみなすことがある。

命題 5.1.6 ([0042]). 関数外延性を仮定する。i を階数、$A : \mathcal{U}(i)$ を型とすると、型 $\mathsf{IsContr}(A)$ は命題である。

証明. 命題 5.1.3 より、$\mathsf{IsContr}(A) \to \mathsf{IsContr}(\mathsf{IsContr}(A))$ を示せばよい。A が可縮であると仮定する。要素 $a_0 : A$ を得る。レトラクトの列

$$\begin{array}{ll} & \mathsf{IsContr}(A) \\ \lhd & \quad \{\text{定義}\} \\ & \sum_{a:A} \prod_{x:A} a = x \\ \lhd & \quad \{A \text{ は可縮}\} \\ & \prod_{x:A} a_0 = x \end{array}$$

を得て、最後の型は命題 3.1.3 と命題 3.4.3 より可縮である。 □

系 5.1.7 ([0043]). 関数外延性を仮定する。i を階数、$A : \mathcal{U}(i)$ を型、$n : \mathsf{TruncLevel}$ を要素とすると、$\mathsf{IsTrunc}(n, A)$ は命題である

証明. 命題 5.1.6 と命題 5.0.6 から、n についての帰納法で示せる。 □

系 5.1.8 ([0044]). 関数外延性を仮定する。i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とすると、$\mathsf{IsEquiv}(f)$ は命題である。 □

命題 5.1.9 ([0054]). 一価性と関数外延性を仮定する。i を階数、$n : \mathsf{TruncLevel}$ を要素とする。$\langle n \rangle\text{-}\mathsf{Type}(i)$ は $\mathsf{suc}(n)$ 型である。

証明. $A, B : \langle n \rangle\text{-}\mathsf{Type}(i)$ を要素とする。一価性と命題 5.1.4 と系 5.1.7 より、同値 $(A = B) \simeq (A.\mathsf{Type} \simeq B.\mathsf{Type})$ を得て、後者は命題 5.0.6 と命題 5.0.7 と系 5.0.9 から n 型である。 □

5.1.1 同値の概念 [004N]

系 5.1.8 より、関数が同値であることは命題であることが分かったが、項 3.6.1 で導入した他の同値の概念が命題であることも示す。

演習 5.1.10 ([004P]). 関数外延性を仮定する。i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。f は同値であると仮定して、次を示せ。

1. 任意の型 $X : \mathcal{U}(i)$ に対して、関数 $\lambda g.f \circ g : (X \to A) \to (X \to B)$ は同値である。
2. 任意の型 $Y : \mathcal{U}(i)$ に対して、関数 $\lambda h.h \circ f : (B \to Y) \to (A \to Y)$ は同値である。

系 5.1.11 ([004Q]). 関数外延性を仮定する。i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とする。f が同値ならば、$\mathsf{LInv}(f)$ と $\mathsf{RInv}(f)$ は可縮である。

証明. 関数外延性から、レトラクト $\mathsf{RInv}(f) \lhd \mathsf{Fiber}(\lambda(g : B \to A).f \circ g, \mathsf{id}(B))$ を得て、右辺は演習 5.1.10 より可縮である。$\mathsf{LInv}(f)$ についても同様である。 □

命題 5.1.12 ([004O]). 関数外延性を仮定する。i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とすると、型 $\mathsf{IsBiinv}(f)$ は命題である。

証明. 命題 5.1.3 より、f が両側可逆であると仮定して $\mathsf{IsBiinv}(f)$ が可縮であることを示せばよいが、命題 3.6.10 より f は同値なので、系 5.1.11 から $\mathsf{IsBiinv}(f)$ が可縮であることが従う。 □

補題 5.1.13 ([004S]). i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数、$a_1, a_2 : A$ を要素とする。f が同値ならば $\mathsf{ap}(f) : a_1 = a_2 \to f(a_1) = f(a_2)$ は同値である。

証明. 定理 3.2.4 を適用する。$\sum_{x:A} f(a_1) = f(x)$ が可縮であることを示せばよいが、この型は $\mathsf{Fiber}(f, f(a_1))$ のレトラクトであることがすぐに分かり、仮定から後者の型は可縮である。 □

命題 5.1.14 ([004R]). 関数外延性を仮定する。i を階数、$A, B : \mathcal{U}(i)$ を型、$f : A \to B$ を関数とすると、型 $\mathsf{IsHAE}(f)$ は命題である。

証明. 命題 5.1.3 より、f が半随伴同値であると仮定して $\mathsf{IsHAE}(f)$ が可縮であることを示せばよい。演習 3.6.12 より f は同値である。レトラクト

$$
\begin{array}{ll}
 & \mathsf{IsHAE}(f) \\
\lhd & \quad \{\text{並び替え}\} \\
 & \sum_{g:B\to A} \sum_{\varepsilon: f\circ g \sim \mathsf{id}(B)} \sum_{\eta: g\circ f \sim \mathsf{id}(A)} \prod_{x:A} f(\eta(x)) = \varepsilon(f(x)) \\
\lhd & \quad \{\text{系 5.1.11。可縮性から適当な } g \text{ と } \varepsilon \text{ を取れる。}\} \\
 & \sum_{\eta: g\circ f \sim \mathsf{id}(A)} \prod_{x:A} f(\eta(x)) = \varepsilon(f(x)) \\
\lhd & \quad \{\text{例 2.3.11}\} \\
 & \prod_{x:A} \sum_{p: g(f(x))=x} f(p) = \varepsilon(f(x))
\end{array}
$$

を得て、最後の型は補題 5.1.13 と命題 3.4.3 より可縮である。 □

5.2 集合 [004B]

定義 5.2.1 ([004C]). i を階数、$A : \mathcal{U}(i)$ を型とする。型 $\mathsf{IsSet}(A) : \mathcal{U}(i)$ を $\mathsf{IsTrunc}(0, A)$ と定義する。$\mathsf{IsSet}(A)$ の要素がある時、A は**集合** (set) であると言う。

命題 5.2.2 ([004D]). i を階数、$A : \mathcal{U}(i)$ を型とする。次の型は論理的に同値である。

1. $\mathsf{IsSet}(A)$
2. $\prod_{x:A} \mathsf{IsContr}(x = x)$
3. $\prod_{x:A} \prod_{z:x=x} \mathsf{refl}(x) = z$ (*Axiom K* と呼ばれる)
4. $\prod_{x_1,x_2:A} \prod_{z_1,z_2:x_1=x_2} z_1 = z_2$ (*uniqueness of identity principle (UIP)* と呼ばれる)

証明. 1 から 2 を示す。A が集合であると仮定する。要素 $x : A$ に対して、$x = x$ は命題である。要素 $\mathsf{refl}(x) : x = x$ があるので、命題 5.1.3 より $x = x$ は可縮である。

2 から 3 は命題 3.1.3 による。

4 において z_1 について帰納法を使うと 3 そのものになるので 3 から 4 が従う。

4 から 1 は命題 5.1.3 による。 □

命題 5.2.3 ([004E]). i を階数、$A : \mathcal{U}(i)$ を型、$E : A \to A \to \mathcal{U}(i)$ を型の族、$r : \prod_{x:A} E(x, x)$ と $f : \prod_{x_1,x_2:A} E(x_1, x_2) \to x_1 = x_2$ を関数とする。$\prod_{x_1,x_2:A} \mathsf{IsProp}(E(x_1, x_2))$ の要素がある時、同値 $\prod_{x_1,x_2}(x_1 = x_2) \simeq E(x_1, x_2)$ を構成でき、特に A は集合である。

証明. r から同一視型の帰納法より関数 $g \prod_{x_1,x_2:A} x_1 = x_2 \to E(x_1, x_2)$ を得る。$E(x_1, x_2)$ が命題であるという仮定から、f と g はレトラクト $E(x_1, x_2) \lhd (x_1 = x_2)$ を定める。よって、定理 3.2.4 より同値 $(x_1 = x_2) \simeq E(x_1, x_2)$ を得る。 □

定理 5.2.4 (Hedberg [2][004H]). i を階数、$A : \mathcal{U}(i)$ を型、$d : \prod_{x_1,x_2:A}(x_1 = x_2) + ((x_1 = x_2) \to \mathbf{0})$ を関数とすると、A は集合である。

証明. 命題 5.2.3 を適用する。$E : A \to A \to \mathcal{U}(i)$ を次のように定義する。$x_1, x_2 : A$ に対して、$T(x_1, x_2) : (x_1 = x_2) + ((x_1 = x_2) \to \mathbf{0}) \to \mathcal{U}(i)$ を $T(x_1, x_2, \mathsf{in}_1(z)) \equiv \mathbf{1}$ と $T(x_2, x_2, \mathsf{in}_2(z)) \equiv \mathbf{0}$ で定義する。$E(x_1, x_2) \equiv T(x_1, x_2, d(x_1, x_2))$ と定義する。例 3.1.2 と命題 5.0.8 と例 5.1.2 から、各 $E(x_1, x_2)$ は命題である。関数 $g : \prod_{z:_} T(x_1, x_2, z) \to x_1 = x_2$ を $g(\mathsf{in}_1(u), w) \equiv u$ と $g(\mathsf{in}_2(v), w) \equiv \mathsf{ind}_0(w, _)$ で定義できるので、関数 $E(x_1, x_2) \to x_1 = x_2$ を得る。最後に、任意の $x : A$ に対して同値 $h : \prod_{z:_} T(x, x, z) \simeq \mathbf{1}$ を $h(\mathsf{in}_1(u)) \equiv \mathsf{id}(\mathbf{1})$ と $h(\mathsf{in}_2(v)) \equiv \mathsf{ind}_0(v(\mathsf{refl}(x)), _)$ と定義できるので、関数 $\prod_{x:A} E(x, x)$

を得る。 □

5.3 切り詰め [004Z]

型は∞グルーポイドという高次元の構造を持つのであったが、その豊富な構造ゆえ型を解析するのは必ずしも容易ではない。高次元の構造を解析する常套手段はより低次元の構造で近似することである。型の n 型による最良の近似を導入する。

規則 5.3.1 ([0050]). i を階数、$A : \mathcal{U}(i)$ を型、$n : \mathsf{TruncLevel}$ を要素とする。

1. n **切り詰め** (n-truncation) $\|A\|_n : \mathcal{U}(i)$ を構成できる。
2. $\|A\|_n$ は n 型である (ことを示す要素を構成できる)。
3. 要素 $a : A$ に対して、要素 $|a|_n : \|A\|_n$ を構成できる。
4. $c : \|A\|_n$ を要素、j を階数、$B : \|A\|_n \to \mathcal{U}(j)$ を型の族、$f : \prod_{x:A} B(|x|_n)$ を関数とする。$\prod_{z:\|A\|_n} \mathsf{IsTrunc}(n, B(z))$ の要素があるなら、要素 $\mathsf{ind}_{\mathsf{Tr}(n)}(c, B, f) : B(c)$ を構成できる。
5. $a : A$ を要素、j を階数、$B : \|A\|_n \to \mathcal{U}(j)$ を型の族、$f : \prod_{x:A} B(|x|_n)$ を関数とする。$\prod_{z:\|A\|_n} \mathsf{IsTrunc}(n, B(z))$ の要素があるなら、$\mathsf{ind}_{\mathsf{Tr}(n)}(|a|_n, B, f) \equiv f(a)$ を定義される。

規則 5.3.1 は帰納的型の規則と類似している。実際、$\|A\|_n$ を高次帰納的型 (4 章) の一種と捉えることもできる。$|\ |_n$ は構成子である。それより下の規則は**帰納法原理**を表すが、帰納法が使える状況は B が n 型の族である場合に制限されている。

命題 5.3.2 ([0051]). i を階数、$A : \mathcal{U}(i)$ を型、$n : \mathsf{TruncLevel}$ を要素とする。次は論理的に同値である。

1. A は n 型である。
2. 関数 $\lambda x.|x|_n : A \to \|A\|_n$ は同値である。
3. A は $\|A\|_n$ のレトラクトである。

証明. 2 から 3 は自明である。3 から 1 は 5.0.5 による。

1 から 2 を示す。A が n 型なので、帰納法より関数 $g : \|A\|_n \to A$ であって任意の $a : A$ に対して $g(|a|_n) \equiv a$ となるものを構成できる。特に、$g \circ (\lambda x.|x|_n) \sim \mathsf{id}(A)$ である。$(\lambda x.|x|_n) \circ g \sim \mathsf{id}(\|A\|_n)$ を示す。各 $z : \|A\|_n$ に対して $|g(c)|_n = z$ は系 5.0.9 より n 型なので、帰納法により $\prod_{x:A} |g(|x|_n)|_n = |x|_n$ を示せばよいがこれは定義から明らかである。 □

$\|A\|_n$ の性質を調べる際には一価性が不可欠である。一価性をどう使うかは次の定

理 5.3.3 にまとめられる。

定理 5.3.3 ([0056]). 一価性と関数外延性を仮定する。i を階数、$A : \mathcal{U}(i)$ を型、$B : A \to \mathcal{U}(i)$ を型の族、$n : \mathsf{TruncLevel}$ を要素とする。任意の $x : A$ に対して $B(x)$ は n 型であると仮定する。命題 5.1.9 より、帰納法で型の族 $T : \|A\|_{\mathsf{suc}(n)} \to \mathcal{U}(i)$ であって、任意の $u : \|A\|_{\mathsf{suc}(n)}$ に対して $T(u)$ は n 型であり、任意の $x : A$ に対して $T(|x|_{\mathsf{suc}(n)}) \equiv B(x)$ であるものを構成できる。命題 5.0.7 と命題 5.0.8 より $\sum_{u:\|A\|_{\mathsf{suc}(n)}} T(u)$ は $\mathsf{suc}(n)$ 型なので、関数 $\lambda z.\mathsf{pair}(|\mathsf{pr}_1(z)|_{\mathsf{suc}(n)}, \mathsf{pr}_2(z)) : (\sum_{x:A} B(x)) \to (\sum_{u:\|A\|_{\mathsf{suc}(n)}} T(u))$ は関数

$$H : \|\textstyle\sum_{x:A} B(x)\|_{\mathsf{suc}(n)} \to (\sum_{u:\|A\|_{\mathsf{suc}(n)}} T(u))$$

を誘導する。この時、関数 H は同値である。

証明. $\sum_{u:\|A\|_{\mathsf{suc}(n)}} T(u)$ が $\|\sum_{x:A} B(x)\|_{\mathsf{suc}(n)}$ と同じ帰納法原理を満たすことを見る。$C : (\sum_{u:\|A\|_{\mathsf{suc}(n)}} T(u)) \to \mathcal{U}(i)$ を型の族とし、各 $C(u)$ は $\mathsf{suc}(n)$ 型であると仮定する。$f : \prod_{z:\sum_{x:A} B(x)} C(\mathsf{pair}(|z|_{\mathsf{suc}(n)}, \mathsf{pr}_2(z)))$ を関数とする。$D : \prod_{u:\|A\|_{\mathsf{suc}(n)}} T(u) \to \mathcal{U}(i)$ を C のカリー化、$g : \prod_{x:A} \prod_{y:B(x)} D(|x|_{\mathsf{suc}(n)}, y)$ を f のカリー化とする。命題 5.0.6 より各 $\prod_{v:D(u)} D(u, v)$ は $\mathsf{suc}(n)$ 型なので、帰納法で関数 $h : \prod_{u:\|A\|_{\mathsf{suc}(n)}} \prod_{v:T(u)} D(u, v)$ を得る。h を逆カリー化すればよい。 □

定理 5.3.3 を使う例として、$\|A\|_n$ の同一視型の特徴付けを与える。n が -2 の場合は自明なので、興味があるのはそれ以外の場合である。

命題 5.3.4 ([0055]). 一価性と関数外延性を仮定する。i を階数、$A : \mathcal{U}(i)$ を型、$a_1, a_2 : A$ を要素、$n : \mathsf{TruncLevel}$ を要素とすると、同値 $(|a_1|_{\mathsf{suc}(n)} = |a_2|_{\mathsf{suc}(n)}) \simeq \|a_1 = a_2\|_n$ を構成できる。

証明. 定理 5.3.3 を型の族 $\lambda x.\|a_1 = x\|_n : A \to \mathcal{U}(i)$ に適用すると、定理 3.2.4 より、$\|\sum_{x:A} \|a_1 = x\|_n\|_{\mathsf{suc}(n)}$ が可縮であることを示せば十分である。要素 $c_1 : \|\sum_{x:A} \|a_1 = x\|_n\|_{\mathsf{suc}(n)}$ を $|\mathsf{pair}(a_1, |\mathsf{refl}(a_1)|_n)|_{\mathsf{suc}(n)}$ と定義する。任意の $w : \|\sum_{x:A} \|a_1 = x\|_n\|_{\mathsf{suc}(n)}$ に対して同一視 $c_1 = w$ を構成する。系 5.0.9 よりこの同一視型は $\mathsf{suc}(n)$ 型なので、帰納法より任意の $x : A$ と $v : \|a_1 = x\|_n$ に対して同一視 $c_1 = |\mathsf{pair}(x, v)|_{\mathsf{suc}(n)}$ を構成すればよい。この同一視型は定義から n 型なので、帰納法より任意の $x : A$ と $y : a_1 = x$ に対して同一視 $c_1 = |\mathsf{pair}(x, |y|_n)|_{\mathsf{suc}(n)}$ を構成すればよいが、同一視型の帰納法で $\mathsf{refl}(c_1)$ を与えればよい。 □

5.4 述語論理 [0057]

命題の概念の導入により、**一階述語論理**を型理論の中で模倣できる。

記法 5.4.1 ([0058]). i を階数とする。

1. 型 $\top : \mathcal{U}(0)$ を $\mathbf{1}$ と定義する。
2. 型 $P, Q : \mathcal{U}(i)$ に対して、型 $P \wedge Q : \mathcal{U}(i)$ を $P \times Q$ と定義する。
3. 型 $\bot : \mathcal{U}(0)$ を $\mathbf{0}$ と定義する。
4. 型 $P, Q : \mathcal{U}(i)$ に対して、型 $P \vee Q : \mathcal{U}(i)$ を $\|P + Q\|_{-1}$ と定義する。
5. 型 $P, Q : \mathcal{U}(i)$ に対して、型 $P \Rightarrow Q : \mathcal{U}(i)$ を $P \to Q$ と定義する。
6. 型 $P : \mathcal{U}(i)$ に対して、型 $\neg P : \mathcal{U}(i)$ を $P \Rightarrow \bot$ と定義する。
7. 型 $P, Q : \mathcal{U}(i)$ に対して、型 $P \Leftrightarrow Q : \mathcal{U}(i)$ を $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$ と定義する。
8. 型 $A : \mathcal{U}(i)$ と型の族 $P : A \to \mathcal{U}(i)$ に対して、型 $\forall_{x:A} P(x) : \mathcal{U}(i)$ を $\prod_{x:A} P(x)$ と定義する。
9. 型 $A : \mathcal{U}(i)$ と型の族 $B : A \to \mathcal{U}(i)$ に対して、型 $\exists_{x:A} B(x) : \mathcal{U}(i)$ を $\|\sum_{x:A} B(x)\|_{-1}$ と定義する。

これらの記法は P や Q が命題である場合に使い、結果も命題であることが分かる。

用語 5.4.2 ([005A]). i を階数、$P : \mathcal{U}(i)$ を型とする。P が命題である場合、「P の要素がある」の代わりに「P は**真**である」という言い方をする。

ただし、特別な公理を課さない限り型理論で模倣できる論理は**直観主義論理**である。特に、命題 P に対して $P \vee \neg P$ が真であるとは限らない。

公理 5.4.3 (排中律 [0059]). **排中律** (law of excluded middle) は任意の階数 i と型 $P : \mathcal{U}(i)$ に対して、P が命題ならば $P \vee \neg P$ が真であることを要請する。

排中律は従来の数学では当たり前のように使われるが、ホモトピー型理論ではほとんど**排中律を仮定しない**。

5.5 構造同一原理 [004V]

3.5 節で構造同一原理の例をいくつか見たが、群や環などの数学的に興味深い構造については後回しにしていた。これは、型理論において「$a_1 = a_2$」は単なる型なので公理を適切に記述するためには 5.1 節の意味での命題の概念が必要だからである。例えば、ある二項演算 $\times$ の結合律を素直に $(a_1 \times a_2) \times a_3 = a_1 \times (a_2 \times a_3)$ と書くと、これは公理というよりも構造の一部になってしまう。結合律を適切に公理として扱うためには、同一視型が命題であることを要請すればよい。

例 5.5.1 ([004W]). i を階数とする。型 $\mathsf{GroupStr}(i) : \mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- Carrier : $\mathcal{U}(i)$
- unit : Carrier
- mul : Carrier → Carrier → Carrier
- inv : Carrier → Carrier

要素 G : GroupStr(i) に対して、型 GroupAxiom(G) : $\mathcal{U}(i)$ を次のレコード型と定義する。

- _ : IsSet(G.Carrier)
- _ : $\prod_{x:G.\mathsf{Carrier}} G.\mathsf{mul}(G.\mathsf{unit}, x) = x$
- _ : $\prod_{x:G.\mathsf{Carrier}} G.\mathsf{mul}(x, G.\mathsf{unit}) = x$
- _ : $\prod_{x_1,x_2,x_3:G.\mathsf{Carrier}} G.\mathsf{mul}(G.\mathsf{mul}(x_1, x_2), x_3) = G.\mathsf{mul}(x_1, G.\mathsf{mul}(x_2, x_3))$
- _ : $\prod_{x:G.\mathsf{Carrier}} G.\mathsf{mul}(G.\mathsf{inv}(x), x) = G.\mathsf{unit}$
- _ : $\prod_{x:G.\mathsf{Carrier}} G.\mathsf{mul}(x, G.\mathsf{inv}(x)) = G.\mathsf{unit}$

型 Group(i) : $\mathcal{U}(\mathsf{suc}(i))$ を $\sum_{X:\mathsf{GroupStr}(i)} \mathsf{GroupAxiom}(X)$ と定義する。Group(i) の要素を (階数 i の) **群** (group) と呼ぶ。群 A, B : Group(i) に対して、同一視型 $A = B$ を計算しよう。命題 5.0.7 と命題 5.0.6 より GroupAxiom(X) は命題であることが分かるので、命題 5.1.4 より同値 $(A = B) \simeq (\mathsf{pr}_1(A) = \mathsf{pr}_1(B))$ を得る。例 3.5.2 と同様に、$\mathsf{pr}_1(A) = \mathsf{pr}_1(B)$ は次のレコード型と同値であることが分かる。

- f : $\mathsf{pr}_1(A).\mathsf{Carrier} \simeq \mathsf{pr}_1(B).\mathsf{Carrier}$
- _ : $f(\mathsf{pr}_1(A).\mathsf{unit}) = \mathsf{pr}_1(B).\mathsf{unit}$
- _ : $\prod_{x_1,x_2:\mathsf{pr}_1(A).\mathsf{Carrier}} f(\mathsf{pr}_1(A).\mathsf{mul}(x_1, x_2)) = \mathsf{pr}_1(B).\mathsf{mul}(f(x_1), f(x_2))$
- _ : $\prod_{x:\mathsf{pr}_1(A).\mathsf{Carrier}} f(\mathsf{pr}_1(A).\mathsf{inv}(x)) = \mathsf{pr}_1(B).\mathsf{inv}(f(x))$

この型の要素はいわゆる**群同型** (group isomorphism) である。GroupAxiom(G) の定義で IsSet(G.Carrier) を落とすと、Group(i) の同一視の特徴付けは演算だけでなく残りの構造も保つ同値というものになってしまう。

ちなみに、群の間の乗法を保つ関数は単位元と逆元も保つことを示せるので、群同型の型は次のレコード型とも同値である。

- f : $\mathsf{pr}_1(A).\mathsf{Carrier} \simeq \mathsf{pr}_1(B).\mathsf{Carrier}$
- _ : $\prod_{x_1,x_2:\mathsf{pr}_1(A).\mathsf{Carrier}} f(\mathsf{pr}_1(A).\mathsf{mul}(x_1, x_2)) = \mathsf{pr}_1(B).\mathsf{mul}(f(x_1), f(x_2))$

例 5.5.2 ([004Y]). i を階数とする。型 RingStr(i) : $\mathcal{U}(\mathsf{suc}(i))$ を次のレコード型と定義する。

- Carrier : $\mathcal{U}(i)$
- zero : Carrier
- plus : Carrier → Carrier → Carrier

- neg : Carrier → Carrier
- one : Carrier
- mul : Carrier → Carrier → Carrier

要素 R : RingStr(i) に対して、型 RingAxiom(R) : $\mathcal{U}(i)$ を次のレコード型と定義する。

- _ : IsSet(R.Carrier)
- _ : $\prod_{x:R.\mathsf{Carrier}} R.\mathsf{plus}(R.\mathsf{zero}, x) = x$
- _ : $\prod_{x_1,x_2,x_3:R.\mathsf{Carrier}} R.\mathsf{plus}(R.\mathsf{plus}(x_1,x_2),x_3) = R.\mathsf{plus}(x_1, R.\mathsf{plus}(x_2,x_3))$
- _ : $\prod_{x_1,x_2:R.\mathsf{Carrier}} R.\mathsf{plus}(x_1,x_2) = R.\mathsf{plus}(x_2,x_1)$
- _ : $\prod_{x:R.\mathsf{Carrier}} R.\mathsf{plus}(x, R.\mathsf{neg}(x)) = R.\mathsf{zero}$
- _ : $\prod_{x:R.\mathsf{Carrier}} R.\mathsf{mul}(R.\mathsf{one}, x) = x$
- _ : $\prod_{x:R.\mathsf{Carrier}} R.\mathsf{mul}(x, R.\mathsf{one}) = x$
- _ : $\prod_{x_1,x_2,x_3:R.\mathsf{Carrier}} R.\mathsf{mul}(R.\mathsf{mul}(x_1,x_2),x_3) = R.\mathsf{mul}(x_1, R.\mathsf{mul}(x_2,x_3))$
- _ : $\prod_{x,y_1,y_2:R.\mathsf{Carrier}} R.\mathsf{mul}(x, R.\mathsf{plus}(y_1,y_2)) = R.\mathsf{plus}(R.\mathsf{mul}(x,y_1), R.\mathsf{mul}(x,y_2))$
- _ : $\prod_{x_1,x_2,y:R.\mathsf{Carrier}} R.\mathsf{mul}(R.\mathsf{plus}(x_1,x_2),y) = R.\mathsf{plus}(R.\mathsf{mul}(x_1,y), R.\mathsf{mul}(x_2,y))$

型 Ring(i) : $\mathcal{U}(\mathsf{suc}(i))$ を $\sum_{X:\mathsf{RingStr}(i)} \mathsf{RingAxiom}(X)$ と定義する。Ring(i) の要素を (階数 i の) **環** (ring) と呼ぶ。環 A, B : Ring(i) に対して、同一視型 $A = B$ は次のレコード型と同値であることが分かる。

- f : $\mathsf{pr}_1(A).\mathsf{Carrier} \simeq \mathsf{pr}_1(B).\mathsf{Carrier}$
- _ : $f(\mathsf{pr}_1(A).\mathsf{zero}) = \mathsf{pr}_1(B).\mathsf{zero}$
- _ : $\prod_{x_1,x_2:\mathsf{pr}_1(A).\mathsf{Carrier}} f(\mathsf{pr}_1(A).\mathsf{plus}(x_1,x_2)) = \mathsf{pr}_1(B).\mathsf{plus}(f(x_1), f(x_2))$
- _ : $\prod_{x:\mathsf{pr}_1(A).\mathsf{Carrier}} f(\mathsf{pr}_1(A).\mathsf{neg}(x)) = \mathsf{pr}_1(B).\mathsf{neg}(f(x))$
- _ : $f(\mathsf{pr}_1(A).\mathsf{one}) = \mathsf{pr}_1(B).\mathsf{one}$
- _ : $\prod_{x_1,x_2:\mathsf{pr}_1(A).\mathsf{Carrier}} f(\mathsf{pr}_1(A).\mathsf{mul}(x_1,x_2)) = \mathsf{pr}_1(B).\mathsf{mul}(f(x_1), f(x_2))$

この型の要素はいわゆる**環同型** (ring isomorphism) である。

第 II 部

数学 [0007]

第 6 章

ホモトピー論 [0005]

第 7 章

圏論 [0004]

参考文献

[1] Benno van den Berg and Richard Garner. "Types are weak ω-groupoids". In: *Proceedings of the London Mathematical Society* 102.2 (2011), pp. 370–394. ISSN: 1460-244X. DOI: 10.1112/plms/pdq026. arXiv: 0812.0298v2.

[2] Michael Hedberg. "A coherence theorem for Martin-Löf's type theory". In: *J. Funct. Programming* 8.4 (1998), pp. 413–436. ISSN: 0956-7968. DOI: 10.1017/S0956796898003153.

[3] Peter LeFanu Lumsdaine. "Weak ω-categories from intensional type theory". In: *Logical Methods in Computer Science* 6.3 (Sept. 2010). ISSN: 18605974. DOI: 10.2168/lmcs-6(3:24)2010. arXiv: 0812.0409v4.

[4] Thomas Streicher. "Investigations into Intensional Type Theory". Habilitation. Ludwig-Maximilians-Universität München, 1993. URL: https://www2.mathematik.tu-darmstadt.de/~streicher/HabilStreicher.pdf.

[HoTT Book] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. URL: http://homotopytypetheory.org/book/.

索引

A 上の $\mathbb{N}$ 代数, 34

n 型, 41
n 切り詰め, 47
$\mathbb{N}$ 準同型写像, 33
$\mathbb{N}$ 代数, 33

$\mathsf{PO}(f,g)$ 準同型写像, 37
$\mathsf{PO}(f,g)$ 代数, 37
p 上の同一視, 36

$\mathbb{S}^1$ 準同型写像, 36
$\mathbb{S}^1$ 代数, 36

暗黙的, 7
一価性, 23
一価性公理, 23
宇宙, 5
階数, 5
可縮, 18
型, 6
型の族, 7
カリー化, 8
環, 51
関数, 6
関数外延性, 24
関数外延性公理, 24
関数型, 6
関数適用, 6
環同型, 51
帰納法原理, 14
逆カリー化, 8
空型, 14
群, 50
群同型, 50
原始再帰, 14
構成子, 14
構造同一原理, 25
恒等関数, 7
合成関数, 7
再帰的定義, 14
始 $\mathbb{N}$ 代数, 33
始 $\mathsf{PO}(f,g)$ 代数, 37
始 $\mathbb{S}^1$ 代数, 36
自然数, 13
自然数型, 13
射影, 8
集合, 46
真, 49
切断, 34
単位型, 7
対, 8
対型, 8
点付き型, 26
同一視, 10
同一視型, 10

同一視型の基本定理, 21
同値, 19
排中律, 49
反射的グラフ, 26
半随伴同値, 31
爆発原理, 15
ホモトピー, 29
マグマ, 26
矛盾, 15
命題, 43
輸送関数, 11
余積, 15
ラムダ抽象, 6
両側可逆, 30
レコード型, 9
レトラクト, 19
論理的に同値, 10

識別子一覧

[0000], 1
[0001], 5
[0002], 17
[0003], 33
[0004], 57
[0005], 55
[0006], 5
[0007], 55
[0009], 5
[000A], 6
[000B], 7
[000C], 10
[000D], 5
[000E], 6
[000F], 6
[000G], 6
[000H], 6
[000I], 7
[000J], 7
[000K], 7
[000L], 8
[000M], 8
[000N], 8
[000O], 9
[000P], 10
[000Q], 7
[000R], 1
[000S], 17
[000T], 18
[000U], 27
[000V], 19
[000W], 10
[000X], 23
[000Y], 23
[000Z], 23
[0010], 7
[0011], 7
[0012], 7
[0013], 7
[0014], 8
[0015], 8
[0016], 8
[0017], 8
[0018], 9
[0019], 9
[001A], 9
[001B], 9
[001C], 11
[001D], 11
[001E], 11
[001F], 12
[001G], 11
[001H], 11
[001I], 12

[001J], 19
[001K], 19
[001L], 18
[001M], 12
[001N], 18
[001O], 18
[001P], 19
[001Q], 19
[001R], 20
[001S], 21
[001T], 10
[001U], 10
[001V], 20
[001W], 21
[001X], 20
[001Y], 24
[001Z], 24
[0020], 24
[0021], 25
[0022], 25
[0023], 25
[0024], 22
[0025], 22
[0026], 23
[0027], 12
[0028], 26
[0029], 24
[002A], 26
[002B], 22
[002C], 22
[002D], 22
[002E], 28
[002F], 29
[002G], 29
[002H], 30
[002I], 29
[002J], 28
[002K], 29
[002L], 27
[002M], 9
[002N], 12
[002O], 12
[002P], 13
[002Q], 13
[002R], 13
[002S], 13
[002T], 13
[002U], 14
[002V], 13
[002W], 14
[002X], 14
[002Y], 14
[002Z], 15
[0030], 14
[0031], 33
[0032], 36
[0033], 36
[0034], 37
[0035], 33
[0036], 33
[0037], 33
[0038], 33
[0039], 34
[003A], 34
[003B], 34
[003C], 35
[003D], 34
[003E], 35
[003F], 35
[003G], 34
[003H], 34
[003I], 35

[003J], 36
[003K], 36
[003L], 36
[003M], 36
[003N], 36
[003O], 36
[003P], 37
[003Q], 37
[003R], 37
[003S], 37
[003T], 38
[003U], 38
[003V], 38
[003W], 41
[003X], 41
[003Y], 41
[003Z], 43
[0040], 43
[0041], 43
[0042], 44
[0043], 44
[0044], 44
[0045], 41
[0046], 41
[0047], 13
[0048], 42
[0049], 43
[004A], 43
[004B], 46
[004C], 46
[004D], 46
[004E], 46
[004F], 42
[004G], 43
[004H], 46
[004I], 30
[004J], 30
[004K], 30
[004L], 31
[004M], 31
[004N], 44
[004O], 45
[004P], 44
[004Q], 45
[004R], 45
[004S], 45
[004T], 31
[004U], 31
[004V], 49
[004W], 49
[004X], 42
[004Y], 50
[004Z], 47
[0050], 47
[0051], 47
[0052], 42
[0053], 41
[0054], 44
[0055], 48
[0056], 48
[0057], 48
[0058], 49
[0059], 49
[005A], 49